

# 5주차 예습과제- 딥러닝 파이토치 교과서 6장-Part 1 (~6.1.4)

## 6장- 합성곱 신경망 II

### 6.1 이미지 분류를 위한 신경망

이미지 분류(image classification) = 이미지 안에 특정 대상이 있는지 / 무엇인지 판단하는 것

#### 6.1.1 LeNet-5

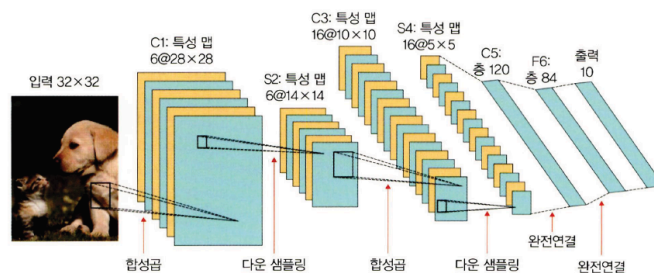
##### 1. LeNet-5란?

- 1995년 얀 르쿤(Yann LeCun), 레옹 보토(Leon Bottou), 요슈아 벤지오(Yosua Bengio), 패트릭 하프너(Patrick Haffner)가 발표
- 수표에 쓴 손글씨 숫자를 인식하는 딥러닝 구조 → 현재 **CNN의 초석**
- 핵심 아이디어: **합성곱(convolutional) + 다운 샘플링(sub-sampling, = 풀링)**을 반복 → 마지막에 **완전연결층**에서 분류

| 기호       | 의미                      |
|----------|-------------------------|
| <b>C</b> | 합성곱층 (Convolution)      |
| <b>S</b> | 풀링층 (Sub-sampling)      |
| <b>F</b> | 완전연결층 (Fully connected) |

##### 원조 LeNet-5 구조

▼ 그림 6-1 LeNet-5



입력: 32×32

| 층         | 연산      | 출력              | 설명                |
|-----------|---------|-----------------|-------------------|
| 입력        | -       | 32×32           | 이미지               |
| <b>C1</b> | 5×5 합성곱 | <b>6@28×28</b>  | 28×28 특성 맵 6개 생성  |
| <b>S2</b> | 다운 샘플링  | <b>6@14×14</b>  | 크기를 14×14로 축소     |
| <b>C3</b> | 5×5 합성곱 | <b>16@10×10</b> | 10×10 특성 맵 16개 생성 |

| 층  | 연산      | 출력      | 설명                    |
|----|---------|---------|-----------------------|
| S4 | 다운 샘플링  | 16@5×5  | 크기를 5×5로 축소           |
| C5 | 5×5 합성곱 | 120@1×1 | 1×1 특성 맵 120개         |
| F6 | 완전연결    | 84      | C5 결과를 유닛(노드) 84개에 연결 |
| 출력 | 완전연결    | 10      | 숫자 0~9 분류             |

## 2. 준비

### tqdm

- 설치: `pip install --user tqdm` 또는 `conda install -c conda-forge tqdm`
- `tqdm` = 아랍어로 "progress(진행 상태)" → 진행 상태를 **바(bar) 형태로 시각화**, 모델 훈련 진행 확인용

### 주요 라이브러리 역할 (코드 6-1)

| 라이브러리                                     | 역할                      |
|-------------------------------------------|-------------------------|
| <code>torchvision.transforms</code>       | 이미지 변환(전처리) 기능 제공       |
| <code>torch.optim</code>                  | 경사 하강법으로 가중치를 구하는 옵티마이저 |
| <code>os</code>                           | 파일 경로 관련 함수 제공          |
| <code>cv2</code> , <code>PIL.Image</code> | 이미지 읽기                  |
| <code>tqdm_notebook</code>                | 진행 상황 시각화 (학습 경과 확인)    |
| <code>random</code>                       | 데이터 섞기                  |

### device 설정

```
device = torch.device("cuda:0" if torch.cuda.is_available() else "cpu")
```

- 파이토치는 텐서플로와 달리 **GPU를 자동 할당하지 않음** → 모델과 데이터에 직접 선언해야 함
- 이 장에서는 **CPU 사용**

## 3. 이미지 전처리 (코드 6-2 `ImageTransform`)

💡 **직관**: 모델은 "규격화된 재료"만 받음. 크기 맞추고(Resize/Crop) → 숫자로 바꾸고(ToTensor) → 값 범위 맞추기(Normalize). **훈련용은 일부러 조금씩 변형(augmentation)** 해서 다양한 상황을 연습시키고, **검증용은 변형 없이** 정직하게 평가.

### train vs val 전처리 비교

| 순서 | train                                                    | val                               |
|----|----------------------------------------------------------|-----------------------------------|
| 1  | <code>RandomResizedCrop(resize, scale=(0.5, 1.0))</code> | <code>Resize(256)</code>          |
| 2  | <code>RandomHorizontalFlip()</code>                      | <code>CenterCrop(resize)</code>   |
| 3  | <code>ToTensor()</code>                                  | <code>ToTensor()</code>           |
| 4  | <code>Normalize(mean, std)</code>                        | <code>Normalize(mean, std)</code> |
| 특징 | <b>무작위 변형 O</b> (데이터 증강)                                 | <b>변형 X</b> (항상 같은 결과)            |

## ① transforms 파라미터 정리

| 기호 | 함수                                | 하는 일                                                                 |
|----|-----------------------------------|----------------------------------------------------------------------|
| Ⓐ  | <code>Compose</code>              | 이미지 변형 방식들의 묶음 (순서대로 적용)                                             |
| Ⓑ  | <code>RandomResizedCrop</code>    | 주어진 크기(224×224)로 조정. scale = 원본의 <b>0.5~1.0(50~100%)</b> 면적을 무작위로 자름 |
| Ⓒ  | <code>RandomHorizontalFlip</code> | 주어진 확률로 수평 반전. 기본값 <b>0.5</b> → 절반은 뒤집고 절반은 그대로                      |
| Ⓓ  | <code>ToTensor</code>             | PIL 이미지 → <code>torch.FloatTensor</code>                             |
| Ⓔ  | <code>Normalize</code>            | 채널별 평균·표준편차로 정규화                                                     |

### Ⓓ ToTensor 상세

|       | PIL로 읽은 이미지          | ToTensor 후                     |
|-------|----------------------|--------------------------------|
| 값 범위  | [0, 255]             | <b>[0.0, 1.0]</b>              |
| 차원 순서 | (높이 H × 너비 W × 채널 C) | <b>(채널 C × 높이 H × 너비 W)</b>    |
| 타입    | 이미지 배열               | <code>torch.FloatTensor</code> |

- `ImageFolder` 등 torchvision 메서드는 이미지를 읽을 때 **PIL** 사용

### Ⓔ Normalize 상세

- 사전 훈련된 모델은 대개 **ImageNet**으로 훈련됨 → 그 데이터의 채널별 평균·표준편차에 맞게 정규화 필요
  - 정규화(normalize): 데이터 범위를 원하는 범위로 제한하는 것 (예: 0~1 사이 값)
- **mean = (0.485, 0.456, 0.406), std = (0.229, 0.224, 0.225)** → ImageNet 이미지의 **RGB 채널** 별 평균·표준편차
- 계산: 출력 = (입력 - mean) / std (채널별)
- ⚠ **OpenCV로 읽으면 RGB가 아닌 BGR** → 채널 순서 주의

## ② `__call__` 메서드

| 메서드                   | 실행 시점                    |
|-----------------------|--------------------------|
| <code>__init__</code> | 인스턴스 <b>생성(초기화)</b> 할 때  |
| <code>__call__</code> | 인스턴스를 <b>함수처럼 호출</b> 할 때 |

- 클래스 객체 자체를 호출하면 `__call__`의 리턴값이 반환됨
- 예: `transform = ImageTransform(224, mean, std)` → `transform(img, 'train')` 하면 train 전처리 적용

## 4. 데이터 불러오기 & 분리 (코드 6-3)

| 구분   | 개수         | 슬라이싱                   |
|------|------------|------------------------|
| 훈련용  | <b>400</b> | <code>[:400]</code>    |
| 검증용  | <b>92</b>  | <code>[400:-10]</code> |
| 테스트용 | <b>10</b>  | <code>[-10:]</code>    |

- 시스템 성능이 좋으면 훈련용 **2만 개** 사용 시 성능 ↑ (예제 파일 주석 해제, 대신 다음 두 줄은 주석 처리)

## 흐름

1. 고양이·개 폴더에서 이미지 경로 리스트 생성 (①)
2. `[*cat, *dog]` 로 두 리스트 합치기 (= 언패킹, 리스트를 풀어서 넣기)
3. 읽을 수 있는 이미지만 남기기 (②)
4. 시드 고정 후 섞어서 분리 (③)

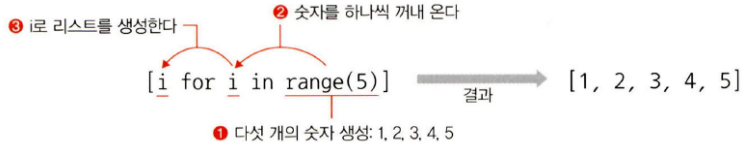
### ① 경로 리스트 만들기 — `sorted([os.path.join(dir, f) for f in os.listdir(dir)])`

| 기호 | 함수                        | 하는 일                                                                                  |
|----|---------------------------|---------------------------------------------------------------------------------------|
| Ⓐ  | <code>sorted</code>       | 정렬된 리스트로 만들어 반환                                                                       |
| Ⓑ  | <code>os.path.join</code> | 경로와 파일명을 결합 / 분할된 경로를 하나로 합침 → <code>../chap06/data/dogs-vs-cats/Cat/cat.0.jpg</code> |
| Ⓒ  | <code>os.listdir</code>   | 지정한 디렉터리 내 모든 파일 리스트 반환                                                               |

- `os.path.join(*['C:\\', 'Temp', 'user'])` → `'C:\\Temp\\user'` (윈도에선 경로가 `\\` 로 표시)

### ② 정상 이미지만 남기기 — `[i for i in images_filepaths if cv2.imread(i) is not None]`

▼ 그림 6-3 for 반복문



| 기호 | 부분                                        | 의미                          |
|----|-------------------------------------------|-----------------------------|
| Ⓐ  | <code>i for</code>                        | 가져온 데이터로 리스트 생성 (리스트 컴프리헨션) |
| Ⓑ  | <code>for i in images_filepaths</code>    | 경로를 하나씩 검색                  |
| Ⓒ  | <code>if cv2.imread(i) is not None</code> | 이미지를 읽어서 조건 검사              |

- 리스트 컴프리헨션 예 (그림 6-3): `[i for i in range(5)]` → `[0, 1, 2, 3, 4]`  
(① 숫자 5개 생성 → ② 하나씩 꺼냄 → ③ i로 리스트 생성)

📌 `cv2.imread()` 는 손상됐거나 이미지가 아닌 파일이면 `None` 을 반환 → 깨진 파일을 걸러내는 조건

### ③ random.seed — 재현성

- 난수는 시드 값(seed value) 으로 생성됨. 시드가 같으면 항상 같은 난수

| <code>np.random.seed()</code> | <code>randint(low=1, high=10, size=10)</code> 결과 |
|-------------------------------|--------------------------------------------------|
| 101                           | [2, 7, 8, 9, 5, 9, 6, 1, 6, 9]                   |
| 100                           | [9, 9, 4, 8, 8, 1, 5, 3, 6, 3]                   |
| 101 (다시)                      | [2, 7, 8, 9, 5, 9, 6, 1, 6, 9] ← 동일!             |

💡 **직관:** 시드 = 카드 섞는 "방법"을 고정. 같은 방법으로 섞으면 매번 같은 순서 → 다시 실행해도 train/val/test가 똑같이 나뉨.

## 5. 테스트 데이터 확인 (코드 6-4, 6-5)

`display_image_grid()` → 테스트 이미지 10장을 2행×5열로 출력. **예측 = 정답이면 초록색, 아니면 빨간색** 제목.

### 핵심 코드 3가지

① `cv2.cvtColor(image, cv2.COLOR_BGR2RGB)` — 색상 변환

- ① 입력 이미지 / ② 변환할 색상: **BGR → RGB**
- 이유: OpenCV는 BGR로 읽고, matplotlib은 RGB로 그림 → 안 바꾸면 색이 이상하게 나옴

② `os.path.normpath(path).split(os.sep)[-2]` — 폴더명을 정답 레이블로

| 기호 | 함수                            | 하는 일                                   | 예시                                                                                                        |
|----|-------------------------------|----------------------------------------|-----------------------------------------------------------------------------------------------------------|
| ①  | <code>os.path.normpath</code> | 경로명 정규화                                | <code>A//B</code> , <code>A/B/</code> , <code>A/.B</code> , <code>A/foo/../B</code> → 모두 <code>A/B</code> |
| ②  | <code>split(os.sep)</code>    | <code>/</code> 또는 <code>\</code> 기준 분할 | <code>c:/temp/user/a.jpg</code> → <code>['c:', 'temp', 'user', 'a.jpg']</code>                            |
|    | <code>[-2]</code>             | 뒤에서 두 번째                               | <code>'user'</code> (우리 데이터: <code>../Cat/cat.0.jpg</code> → <code>'Cat'</code> )                         |

③ `predicted_labels[i] if predicted_labels else true_label` — 삼항 연산자

- ① `predicted_labels`에 값이 **있으면** 그 값 사용
- ② **없으면** `true_label` 사용 (지금은 예측 전이라 전부 초록색)

## 6. Dataset & DataLoader

### 6-1. 개념

💡 **Dataset = 도서관 목록표, DataLoader = 사서.** Dataset은 책(이미지)을 미리 다 꺼내 두지 않고 **위치만 기록해** 두고, 요청이 오면 그때 꺼냄. DataLoader는 책을 **32권씩 섞어서 묶어** 가져다 줌.

| 단계          | 담당                               | 하는 일                                   |
|-------------|----------------------------------|----------------------------------------|
| ① 경로 정의     | <code>Dataset.__init__</code>    | 데이터셋이 클 수 있으므로 전체를 읽지 않고 <b>경로만 저장</b> |
| ② 데이터 읽기    | <code>Dataset.__getitem__</code> | 이미지를 실제로 읽어 오 (어디서 가져올지 결정)            |
| ③ 메모리로 불러오기 | <code>DataLoader</code>          | 한꺼번에가 아니라 <b>배치 크기만큼 분할</b> 해서 가져옴     |

- 여기서 데이터셋 = `torch.utils.data.Dataset`
- 레이블: **고양이 = 0, 개 = 1**

### 6-2. `DogvsCatDataset` 클래스 (코드 6-6)

| 메서드                                                | 역할                                                 |
|----------------------------------------------------|----------------------------------------------------|
| <code>__init__(file_list, transform, phase)</code> | 파일 목록, 전처리 방법, 'train'/'val' 저장                    |
| <code>__len__</code>                               | 데이터셋 전체 길이 반환                                      |
| <code>__getitem__(idx)</code>                      | idx번째 이미지를 열고 → 전처리 적용 → 레이블 추출 → (이미지 텐서, 레이블) 반환 |

#### ① 레이블 추출 — `img_path.split('/')[-1].split('.')[0]`

| 기호 | 부분                           | 결과                                                             |
|----|------------------------------|----------------------------------------------------------------|
| Ⓐ  | <code>img_path</code>        | <code>E:/torch/chap06/data/dogs-vs-cats/Dog/dog.113.jpg</code> |
| Ⓑ  | <code>.split('/')[-1]</code> | / 기준 분리 후 마지막 → <code>dog.113.jpg</code>                       |
| Ⓒ  | <code>.split('.')[0]</code>  | . 기준 분리 후 첫 번째 → <code>dog</code> → 레이블 1                      |

📁 윈도우 경로가 \ 로 저장되면 `split('/')` 이 안 잘릴 수 있음 → `os.path.basename()` 이 더 안전

### 6-3. 변수 값 & 데이터셋 정의 (코드 6-7, 6-8)

| 변수         | 값                     |
|------------|-----------------------|
| size       | 224                   |
| mean       | (0.485, 0.456, 0.406) |
| std        | (0.229, 0.224, 0.225) |
| batch_size | 32                    |

- `train_dataset` ← phase='train' (train 전처리 적용)
- `val_dataset` ← phase='val' (val 전처리 적용)
- 첫 번째 훈련 데이터 확인 결과: `torch.Size([3, 224, 224])`, 레이블 1  
→ 컬러(3채널) 224×224 이미지, 개가 포함된 이미지 (랜덤이라 책과 다를 수 있음)

### 6-4. DataLoader (코드 6-9)

`DataLoader(train_dataset, batch_size=batch_size, shuffle=True)`

- 파이토치의 데이터로더 = 배치 관리 담당. 한 번에 다 불러오면 메모리 부담 → 그룹으로 쪼개서 조금씩

| 기호 | 파라미터                       | 의미                                    |
|----|----------------------------|---------------------------------------|
| Ⓐ  | <code>train_dataset</code> | 데이터를 불러올 데이터셋                         |
| Ⓑ  | <code>batch_size</code>    | 한 번에 불러올 데이터 크기 → 32개씩                |
| Ⓒ  | <code>shuffle</code>       | 가져올 때 무작위로 섞기 (train=True, val=False) |

- `dataloader_dict = {'train': ..., 'val': ...}` → 훈련·검증 로더를 하나로 묶음
- `iter()` + `next()` 로 배치 하나 확인 결과:
  - `torch.Size([32, 3, 224, 224])` = (배치, 채널, 높이, 너비)
  - 레이블 텐서 32개 (0과 1이 섞여 있음)

## 7. 모델 정의 (코드 6-10 LeNet )

### 7-1. 층 구성과 출력 형태

| 층        | 정의                                                        | 출력 형태 (C, H, W)       | 계산                       |
|----------|-----------------------------------------------------------|-----------------------|--------------------------|
| 입력       | -                                                         | (3, 224, 224)         | 컬러 이미지                   |
| cnn1     | <code>Conv2d(3→16, kernel=5, stride=1, padding=0)</code>  | <b>(16, 220, 220)</b> | $(224-5+0)/1+1 = 220$    |
| relu1    | <code>ReLU()</code>                                       | (16, 220, 220)        | 형태 변화 없음                 |
| maxpool1 | <code>MaxPool2d(kernel=2)</code>                          | <b>(16, 110, 110)</b> | $220/2$                  |
| cnn2     | <code>Conv2d(16→32, kernel=5, stride=1, padding=0)</code> | <b>(32, 106, 106)</b> | $(110-5)/1+1$            |
| relu2    | <code>ReLU()</code>                                       | (32, 106, 106)        | -                        |
| maxpool2 | <code>MaxPool2d(kernel=2)</code>                          | <b>(32, 53, 53)</b>   | $106/2$                  |
| view     | 펼치기 (flatten)                                             | <b>89,888</b>         | $32 \times 53 \times 53$ |
| fc1      | <code>Linear(32*53*53, 512)</code>                        | <b>512</b>            | -                        |
| fc2      | <code>Linear(512, 2)</code>                               | <b>2</b>              | 개/고양이                    |
| output   | <code>Softmax(dim=1)</code>                               | 2 (확률, 합=1)           | -                        |

- **forward** 순서: cnn1 → relu1 → maxpool1 → cnn2 → relu2 → maxpool2 → view → fc1 → fc2 → output

💡 `out.view(out.size(0), -1)` = 3차원 블록( $32 \times 53 \times 53$ )을 **한 줄로 펴기**. `out.size(0)` = 배치 크기 유지, `-1` = "나머지는 알아서 계산" → 완전연결층에 넣기 위해 1차원으로 바꾸는 것

### 7-2. 출력 크기 공식 (5장 복습)

**Conv2d:** 출력 크기 =  $(W - F + 2P) / S + 1$

| 기호 | 의미                            |
|----|-------------------------------|
| W  | 입력 데이터 크기 (input_volume_size) |
| F  | 커널 크기 (kernel_size)           |
| P  | 패딩 크기 (padding_size)          |
| S  | 스트라이드 (strides)               |

**MaxPool2d:** 출력 크기 =  $IF / F$

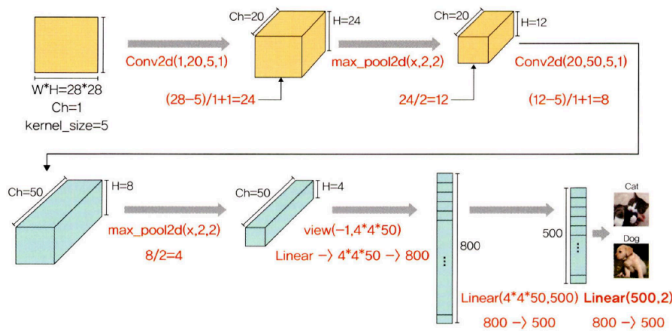
| 기호 | 의미                                                 |
|----|----------------------------------------------------|
| IF | 입력 필터 크기 (input_filter_size, = 바로 앞 Conv2d의 출력 크기) |
| F  | 커널 크기 (kernel_size)                                |

💡 MaxPool2d는 stride를 안 주면 **stride = kernel\_size**가 기본 → 그래서 단순히 나누기로 계산됨

### 7-3. 크기 계산 연습 (입력 $28 \times 28$ , Ch=1)

| 단계 | 연산                   | 계산                     | 결과                   |
|----|----------------------|------------------------|----------------------|
| 1  | Conv2d(1, 20, 5, 1)  | $(28-5)/1+1$           | Ch=20, H= <b>24</b>  |
| 2  | max_pool2d(x, 2, 2)  | $24/2$                 | Ch=20, H= <b>12</b>  |
| 3  | Conv2d(20, 50, 5, 1) | $(12-5)/1+1$           | Ch=50, H= <b>8</b>   |
| 4  | max_pool2d(x, 2, 2)  | $8/2$                  | Ch=50, H= <b>4</b>   |
| 5  | view(-1, 4×4×50)     | $4 \times 4 \times 50$ | <b>800</b>           |
| 6  | Linear(800, 500)     | $800 \rightarrow 500$  | <b>500</b>           |
| 7  | Linear(500, 2)       | $500 \rightarrow 2$    | <b>2 (Cat / Dog)</b> |

♥ 그림 6-5 합성곱층과 풀링층의 크기(형태) 계산



## 8. 모델 구조 & 파라미터 확인 (코드 6-11 ~ 6-13)

### torchsummary

- 케라스(Keras)와 같은 형태로 모델 구조를 보여 주는 라이브러리 (`print(model)` 이 한눈에 안 들어올 때)
- 설치: `pip install torchsummary`
- 사용: `summary(model, input_size=(3,224,224))`
  - ㉠ 모델 네트워크 / ㉡ 입력 크기 (채널, 너비, 높이)

### summary 결과

| Layer (type) | Output Shape       | Param #    |
|--------------|--------------------|------------|
| Conv2d-1     | [-1, 16, 220, 220] | 1,216      |
| ReLU-2       | [-1, 16, 220, 220] | 0          |
| MaxPool2d-3  | [-1, 16, 110, 110] | 0          |
| Conv2d-4     | [-1, 32, 106, 106] | 12,832     |
| ReLU-5       | [-1, 32, 106, 106] | 0          |
| MaxPool2d-6  | [-1, 32, 53, 53]   | 0          |
| Linear-7     | [-1, 512]          | 46,023,168 |
| Linear-8     | [-1, 2]            | 1,026      |
| Softmax-9    | [-1, 2]            | 0          |

| 항목                              | 값          |
|---------------------------------|------------|
| Total params                    | 46,038,242 |
| Trainable params                | 46,038,242 |
| Non-trainable params            | 0          |
| Input size (MB)                 | 0.57       |
| Forward/backward pass size (MB) | 19.47      |
| Params size (MB)                | 175.62     |
| Estimated Total Size (MB)       | 195.67     |

💡 `-1` = 배치 크기 자리 (아직 정해지지 않음)

### 🔍 파라미터 수 직접 계산 (가중치 + 편향)

| 층                        | 계산                                      | 결과         |
|--------------------------|-----------------------------------------|------------|
| Conv2d-1                 | $(3 \times 5 \times 5) \times 16 + 16$  | 1,216      |
| Conv2d-4                 | $(16 \times 5 \times 5) \times 32 + 32$ | 12,832     |
| Linear-7                 | $89,888 \times 512 + 512$               | 46,023,168 |
| Linear-8                 | $512 \times 2 + 2$                      | 1,026      |
| ReLU / MaxPool / Softmax | 학습할 값 없음                                | 0          |

💡 **포인트:** 전체의 **99.9%**가 **fc1** 하나에 몰려 있음. 합성곱층은 커널을 재사용(**가중치 공유**)해서 파라미터가 적지만, 완전연결층은 모든 입력-출력 쌍마다 가중치가 필요하기 때문.

### 학습 가능한 파라미터 수 (코드 6-13)

- `sum(p.numel() for p in model.parameters() if p.requires_grad)`
  - `p.numel()` = 텐서 원소 개수 / `p.requires_grad` = 학습 대상 여부
- 결과: **46,038,242개** → 많아서 학습 시간이 길 것으로 예상 → 캐글 이미지 전체가 아닌 **일부만** 사용한 이유

## 9. 옵티마이저 & 손실 함수 (코드 6-14, 6-15)

- 옵티마이저: `optim.SGD(model.parameters(), lr=0.001, momentum=0.9)`
- 손실 함수: `nn.CrossEntropyLoss()`

### 모멘텀 SGD

- SGD에 **관성**을 추가한 것. 매번 기울기를 구하지만, 가중치 수정 전에 **이전 수정 방향(+, -)**을 참고하여 같은 방향으로 일정 비율만 수정

💡 언덕을 굴러 내려가는 공. 그냥 SGD는 매 걸음 경사만 보고 지그재그로 비틀거리지만, 모멘텀은 **굴러오던 속도(관성)**가 있어서 같은 방향이면 더 빨라지고 진동은 상쇄됨.

| 기호 | 파라미터                            | 의미                                                  |
|----|---------------------------------|-----------------------------------------------------|
| ㉠  | <code>model.parameters()</code> | 업데이트할 파라미터 = 가중치(weight)와 바이어스(bias), 총 46,038,242개 |
| ㉡  | <code>lr=0.001</code>           | 학습률. 가중치를 얼마나 크게 변경할지                               |
| ㉢  | <code>momentum=0.9</code>       | SGD를 적절한 방향으로 가속하며 흔들림(진동)을 줄임                      |

- `model.to(device)`, `criterion.to(device)` → 모델 파라미터와 손실 함수를 device(여기선 CPU)에 할당
- 파이토치는 GPU 할당이 자동이 아니므로 직접 지정

## 10. 모델 학습 (코드 6-16 `train_model`)

### 10-1. 한 epoch의 흐름

| 단계         | train                      | val                       |
|------------|----------------------------|---------------------------|
| 모드 설정      | <code>model.train()</code> | <code>model.eval()</code> |
| 예측         | O                          | O                         |
| 오차 계산      | O                          | O                         |
| 역전파·가중치 갱신 | O                          | X                         |
| 정확도 기록     | O                          | O + 최고 정확도면 가중치 저장        |

### 10-2. 핵심 코드 정리

| 코드                                                               | 의미                                           |
|------------------------------------------------------------------|----------------------------------------------|
| <code>model.train() / model.eval()</code>                        | 학습 모드 / 평가 모드 전환 (Dropout·BatchNorm 동작이 달라짐) |
| <code>for inputs, labels in tqdm(data_loader_dict[phase])</code> | 배치 단위로 데이터 꺼내기 + 진행 바 표시                     |
| <code>optimizer.zero_grad()</code>                               | 역전파 전에 기울기를 0으로 초기화 (파이토치는 기울기를 누적하므로)       |
| <code>torch.set_grad_enabled(phase == 'train')</code>            | train일 때만 기울기 계산 ON                          |
| <code>_, preds = torch.max(outputs, 1)</code>                    | 클래스 방향 최댓값의 인덱스 = 예측 클래스                     |
| <code>loss = criterion(outputs, labels)</code>                   | 손실 함수로 오차 계산                                 |
| <code>loss.backward()</code>                                     | 학습 가능한 모든 파라미터의 기울기 계산                       |
| <code>optimizer.step()</code>                                    | 기울기로 파라미터 갱신                                 |
| <code>epoch_corrects += torch.sum(preds == labels.data)</code>   | 정답과 예측이 일치한 개수 누적                            |
| <code>epoch_loss / len(dataset)</code>                           | 최종 오차 = 오차 합 ÷ 데이터 개수                        |
| <code>epoch_corrects.double() / len(dataset)</code>              | 최종 정확도 = 맞힌 개수 ÷ 데이터 개수                      |
| <code>best_model_wts = model.state_dict()</code>                 | 검증 정확도가 최고일 때 가중치 저장                         |

💡 학습 3종 세트: `zero_grad()` → `backward()` → `step()` = "칠판 지우기 → 틀린 이유 계산 → 고치기"

### 10-3. 왜 `loss.item() * inputs.size(0)` ? — reduction 파라미터

- `nn.CrossEntropyLoss()` 는 `reduction='mean'` 이 기본값 (그래서 따로 명시 안 함)
- 'mean' = 오차들의 평균을 반환 → 즉  $\text{loss} = \text{배치 전체 오차} \div \text{배치 크기}$
- 그래서 배치 크기를 다시 곱해 배치 오차의 합으로 복원 → 마지막에 전체 데이터 개수로 나눔

💡 왜 굳이? 마지막 배치는 크기가 다름 ( $400 \div 32 = 12\text{배치} + 16\text{개짜리 } 1\text{배치}$ ). 평균끼리 다시 평균내면 작은 배치가 과하게 반영되므로, 합으로 되돌린 뒤 전체 개수로 나눠야 정확함.

## 11. 학습 결과 (코드 6-17, num\_epoch = 10)

| Epoch | train Loss | train Acc | val Loss | val Acc |
|-------|------------|-----------|----------|---------|
| 1/10  | 0.6971     | 0.4875    | 0.7125   | 0.4457  |
| 2/10  | 0.6890     | 0.5200    | 0.6837   | 0.5978  |
| ...   | ...        | ...       | ...      | ...     |
| 9/10  | 0.6531     | 0.6450    | 0.6569   | 0.5870  |
| 10/10 | 0.6562     | 0.5975    | 0.6507   | 0.6304  |

- 배치 수: train **13** ( $400 \div 32$ ), val **3** ( $92 \div 32$ ) / train 배치당 약 1.4~1.6초
- Training complete in 4m 6s
- Best val Acc: 0.641304 → 최고 약 **64%**, 높지 않은 결과 → 더 정확하려면 데이터셋을 늘려서 테스트 권장

### 💡 왜 낮을까?

- 훈련 데이터 **400장** vs 파라미터 **4,600만 개** → 데이터에 비해 모델이 너무 큼
- 랜덤 추측이 50%이므로 64%는 "조금 배운" 수준
- 모델 끝 `Softmax` + `CrossEntropyLoss` (내부에서 log-softmax 수행) → **소프트맥스가 두 번 적용되어 기울기가 약해짐**. 정석은 모델 출력을 로짓 그대로 두고 `CrossEntropyLoss`만 사용
- CPU, lr=0.001, 10 epoch로 짧은 학습

## 12. 모델 테스트 (코드 6-18)

- 테스트 데이터셋으로 정확도 측정 → 결과를 데이터 프레임에 담아 CSV로 저장
- 테스트이므로 `model.eval()` 사용

| 코드                                                        | 의미                                                                               |
|-----------------------------------------------------------|----------------------------------------------------------------------------------|
| <code>with torch.no_grad():</code>                        | 역전파 중 변화도(기울기)를 계산할 필요 없음 → <b>훈련과의 가장 큰 차이점</b>                                 |
| <code>test_path.split('/')[ -1 ].split('.')[ 1 ]</code>   | <code>dog.113.jpg</code> → <b>113</b> (이미지 번호를 id로 사용)                           |
| <code>transform(img, phase='val')</code>                  | 테스트에도 <b>변형 없는 val 전처리</b> 적용                                                    |
| ① <code>img.unsqueeze(0)</code>                           | (3, 224, 224) → <b>(1, 3, 224, 224)</b> . 모델은 배치 차원을 기대하므로 "배치 크기 1" 차원을 맨 앞에 추가 |
| ② <code>F.softmax(outputs, dim=1)[ :, 1 ].tolist()</code> | 확률로 변환 후 <b>인덱스 1 = 개일 확률</b> 만 꺼내서 리스트로                                         |
| <code>id_list.append, pred_list.append</code>             | 이미지 번호와 개일 확률 저장                                                                 |

## 12-1. 결과를 데이터 프레임 → CSV로 저장

| 코드                                                             | 의미                                                              |
|----------------------------------------------------------------|-----------------------------------------------------------------|
| <code>pd.DataFrame({'id': id_list, 'label': pred_list})</code> | 테스트 데이터셋의 예측 결과인 id와 레이블(label)을 데이터 프레임에 저장                    |
| <code>res.sort_values(by='id', inplace=True)</code>            | id 기준 정렬 ( <code>inplace=True</code> = 원본 자체를 수정)               |
| <code>res.reset_index(drop=True, inplace=True)</code>          | 정렬 후 뒤섞인 인덱스를 0부터 다시 매김 ( <code>drop=True</code> = 기존 인덱스는 버림)  |
| <code>res.to_csv('../chap06/data/LeNet', index=False)</code>   | 데이터 프레임을 CSV 파일로 저장 ( <code>index=False</code> = 인덱스 열은 저장 안 함) |

📄 파일명에 `.csv` 확장자가 없어서 `LeNet` 이라는 이름으로 저장됨 → `LeNet.csv` 로 쓰는 게 깔끔

## 12-2. ① `torch.unsqueeze` — 차원 추가

- 텐서에 **차원을 추가**할 때 사용. `(0)` = 차원이 추가될 위치

| 원래 형태         | 추가 위치 | 결과 형태                   | 해석                  |
|---------------|-------|-------------------------|---------------------|
| (3)           | 0     | <b>(1, 3)</b>           | 행 1개, 열 3개          |
| (2, 2)        | 0     | <b>(1, 2, 2)</b>        | 채널 1개, 행 2개, 열 2개   |
| (2, 2)        | 1     | <b>(2, 1, 2)</b>        | 채널 2개, 행 1개, 열 2개   |
| (2, 2)        | 2     | <b>(2, 2, 1)</b>        | 채널 2개, 행 2개, 열 1개   |
| (3, 224, 224) | 0     | <b>(1, 3, 224, 224)</b> | ← 이번 코드: 배치 크기 1 추가 |

💡 반대 기능 = `squeeze()` : 크기가 1인 차원을 제거

## 12-3. ② `F.softmax(outputs, dim=1)[:, 1].tolist()`

- 소프트맥스(softmax)** = 지정된 차원(dim)을 따라 텐서의 요소(개별 값)가 **(0, 1)** 범위에 있고 **합계가 1**이 되도록 크기를 다시 조정

| 기호 | 부분                                     | 의미                                                      |
|----|----------------------------------------|---------------------------------------------------------|
| ①  | <code>F.softmax(outputs, dim=1)</code> | outputs에 softmax 적용 → <b>각 행의 합이 1</b>                  |
| ②  | <code>[:, 1]</code>                    | ①의 값 중 <b>모든 행(:)</b> 에서 <b>두 번째 칼럼(1번 인덱스)</b> = 개일 확률 |
| ③  | <code>.tolist()</code>                 | 배열을 <b>리스트</b> 로 변환                                     |

## 인덱스는 0부터 시작

| 인덱스 | 0  | 1  | 2  | 3  | 4  |
|-----|----|----|----|----|----|
| 배열  | 10 | 11 | 12 | 13 | 14 |

예시로 보면:

|                            | 인덱스 0 (cat) | 인덱스 1 (dog) |
|----------------------------|-------------|-------------|
| outputs (softmax 후)        | 0.41        | <b>0.59</b> |
| <code>[:, 1]</code> 로 꺼낸 값 |             | <b>0.59</b> |

## 12-4. 실행 결과

- 10/10 [00:00<00:00, 22.46it/s] → CSV 저장 함수라 결과 자체는 큰 의미 없음, 처리가 완료됐다는 뜻

## 13. 예측 결과 확인 (코드 6-19 ~ 6-21)

### 13-1. 예측 결과 호출 (코드 6-19) — `res.head(10)`

- `head(10)` = DataFrame의 앞에서 10개 값 출력

|   | id  | label    |
|---|-----|----------|
| 0 | 115 | 0.412220 |
| 1 | 116 | 0.422235 |
| 2 | 162 | 0.468849 |
| 3 | 186 | 0.480325 |
| 4 | 190 | 0.350170 |
| 5 | 194 | 0.545277 |
| 6 | 207 | 0.550946 |
| 7 | 39  | 0.503651 |
| 8 | 42  | 0.507188 |
| 9 | 99  | 0.590979 |

- 예측 결과 레이블(label) > 0.5 → 개(dog), < 0.5 → 고양이(cat)
- 위 결과 기준: 115·116·162·186·190 → 고양이 / 194·207·39·42·99 → 개

💡 **관찰 1:** 확률이 전부 0.35~0.59 → 0.5 근처에 몰려 있음 = 모델이 **확신 없이** 거의 찍는 수준 (val 정확도 64%와 일치)

💡 **관찰 2:** id 정렬이 115, 116, ... 207, **39, 42, 99** 순서인 이유 → id가 파일명에서 잘라낸 **문자열**이라 사전순 정렬됨 ('3' < '4' < '9'). 숫자로 정렬하려면 `int(_id)` 로 바꿔 저장

### 13-2. 예측 결과 시각화 함수 (코드 6-20)

- 앞의 `display_image_grid` 와 구조는 같고, 제목을 예측 결과로 표시하도록 수정

| 코드                                                              | 의미                                   |
|-----------------------------------------------------------------|--------------------------------------|
| <code>class_ = {0:'cat', 1:'dog'}</code>                        | 개와 고양이에 대한 클래스 정의 (숫자 → 이름 변환용 딕셔너리) |
| <code>a = random.choice(res['id'].values)</code>                | 데이터 프레임의 id 칼럼에서 임의로 데이터를 가져옴        |
| <code>label = res.loc[res['id'] == a, 'label'].values[0]</code> | id가 a인 행의 label 값(확률)을 꺼냄            |
| <code>if label &gt; 0.5: label = 1 / else: label = 0</code>     | 0.5보다 크면 개(1), 작으면 고양이(0)            |
| <code>set_title(class_[label])</code>                           | 'cat' 또는 'dog'를 이미지 제목으로 표시          |

### 13-3. 예측 결과 이미지 출력 (코드 6-21)

- `display_image_grid(test_images_filepaths)` → 전달하는 파라미터는 **테스트 데이터셋**
- 그림 6-7: 테스트 이미지 10장에 cat / dog 예측 제목이 붙어 출력

| 위치 | 1행  | 2행  |
|----|-----|-----|
| 1열 | cat | cat |
| 2열 | dog | dog |
| 3열 | cat | dog |
| 4열 | dog | cat |
| 5열 | cat | cat |

- 그림 6-4(정답)와 비교하면 1행 4열(실제 고양이 → dog), 2행 4열(실제 개 → cat) 등이 틀림

#### 13-4. 결론

- 예측력이 좋지 않음 → **극히 일부 데이터**로 학습했기 때문
- 캐글 데이터 전체를 쓰면 성능은 높아지지만, **CPU로는 훈련 시간이 며칠** 걸릴 수도 있음

### 14. 6.1.2 AlexNet

#### 14-1. AlexNet이란?

- **ImageNet** 영상 데이터베이스를 기반으로 한 화상 인식 대회 **ILSVRC 2012**에서 **우승**한 CNN 구조

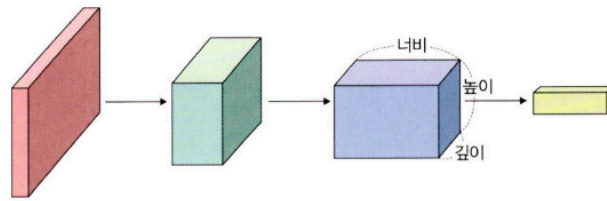
#### 14-2. 먼저 이해할 것: CNN은 3차원 구조

💡 이미지는 종이 한 장(2D)이 아니라 **여러 장을 겹친 두꺼운 책(3D)**. 컬러 이미지는 R/G/B 세 장이 겹친 책이고, 합성곱을 거칠 때마다 "특징 맵" 장수가 늘어나며 책이 **얇고(작아지고) 두꺼워짐(깊어짐)**.

| 축                 | 의미        |
|-------------------|-----------|
| 너비 (width)        | 이미지 가로 크기 |
| 높이 (height)       | 이미지 세로 크기 |
| <b>깊이 (depth)</b> | 채널 수      |

- 이미지를 다루기 때문에 기본적으로 **3차원 데이터**
- 색상이 있는 이미지는 **R/G/B 3개 성분** → 시작 깊이 = 3
- 합성곱을 거치면서 특성 맵이 만들어지고 → 그에 따라 **중간 영상의 깊이가 달라짐**  
(예: LeNet 코드에서 3 → 16 → 32)

♥ 그림 6-8 CNN 구조



### 14-3. AlexNet 구성

| 항목          | 내용                                         |
|-------------|--------------------------------------------|
| 합성곱층        | <b>5개</b>                                  |
| 완전연결층       | <b>3개</b>                                  |
| 마지막 완전연결층   | 카테고리 <b>1000개</b> 분류 → <b>소프트맥스</b> 활성화 함수 |
| 특징          | <b>GPU 두 개 기반 병렬 구조</b>                    |
| LeNet-5와 비교 | 병렬 구조인 점을 제외하면 <b>크게 다르지 않음</b>            |

|       | LeNet-5         | AlexNet          |
|-------|-----------------|------------------|
| 발표    | 1995            | 2012 (ILSVRC 우승) |
| 목적    | 손글씨 숫자 (10개)    | ImageNet (1000개) |
| 합성곱층  | 3개 (C1, C3, C5) | 5개               |
| 완전연결층 | 2개 (F6, 출력)     | 3개               |
| 구조    | 단일              | GPU 2개 병렬        |

### 14-4. AlexNet 구조 (그림 6-9, 표 6-2)

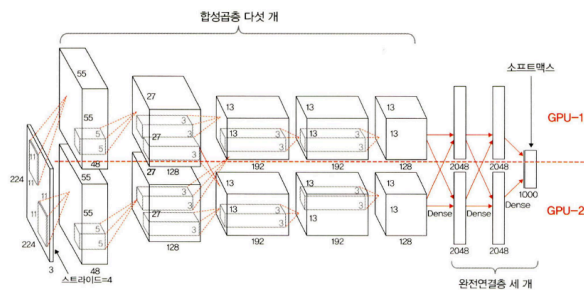
- 그림 6-9: **합성곱층 5개** → **완전연결층 3개** → 소프트맥스. 네트워크가 위(**GPU-1**) / 아래(**GPU-2**) 두 갈래로 나뉘어 병렬 처리됨
- 합성곱층의 활성화 함수 = **렐루(ReLU)**

| 계층 유형  | 특성 맵 | 크기      | 커널 크기 | 스트라이드 | 활성화 함수 |
|--------|------|---------|-------|-------|--------|
| 이미지    | 1    | 227×227 | –     | –     | –      |
| 합성곱층   | 96   | 55×55   | 11×11 | 4     | ReLU   |
| 최대 풀링층 | 96   | 27×27   | 3×3   | 2     | –      |
| 합성곱층   | 256  | 27×27   | 5×5   | 1     | ReLU   |
| 최대 풀링층 | 256  | 13×13   | 3×3   | 2     | –      |
| 합성곱층   | 384  | 13×13   | 3×3   | 1     | ReLU   |
| 합성곱층   | 384  | 13×13   | 3×3   | 1     | ReLU   |
| 합성곱층   | 256  | 13×13   | 3×3   | 1     | ReLU   |
| 최대 풀링층 | 256  | 6×6     | 3×3   | 2     | –      |
| 완전연결층  | –    | 4096    | –     | –     | ReLU   |

| 계층 유형 | 특성 맵 | 크기   | 커널 크기 | 스트라이드 | 활성화 함수 |
|-------|------|------|-------|-------|--------|
| 완전연결층 | -    | 4096 | -     | -     | ReLU   |
| 완전연결층 | -    | 1000 | -     | -     | 소프트맥스  |

| 항목        | 내용                                        |
|-----------|-------------------------------------------|
| 학습 가능한 변수 | 총 <b>6,600만 개</b> (p.273에서는 6,000만 개로 언급) |
| 입력        | <b>227×227×3</b> 크기의 RGB 이미지              |
| 출력        | 각 클래스(카테고리)에 해당하는 <b>1000×1 확률 벡터</b>     |

▼ 그림 6-9 AlexNet 구조

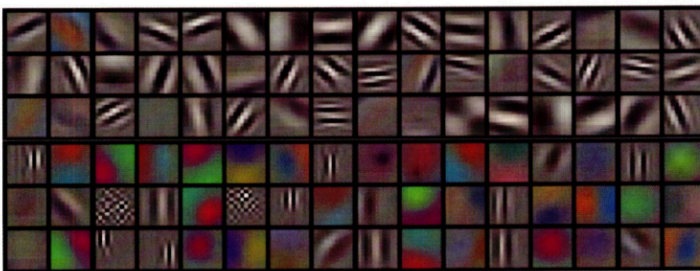


#### 14-5. 첫 번째 합성곱층 & GPU 역할 분담

- 첫 번째 합성곱층 커널 = **11×11×3**, 스트라이드 **4**, 특성 맵 **96개** → 출력 **55×55×96**
- 첫 번째 계층을 거치면서

| GPU          | 주로 학습되는 커널                                           |
|--------------|------------------------------------------------------|
| <b>GPU-1</b> | 컬러와 <b>상관없는</b> 정보 추출 (그림 6-10 위쪽: 흑백 줄무늬·모서리 같은 패턴) |
| <b>GPU-2</b> | 컬러와 <b>관련된</b> 정보 추출 (그림 6-10 아래쪽: 색 덩어리 패턴)         |

▼ 그림 6-10 AlexNet GPU-1 · 2 적용 결과



💡 두 명이 그림을 나눠 보는 것. 한 명은 "선과 모양"만, 다른 한 명은 "색깔"만 담당 → 시키지 않았는데도 자연스럽게 역할이 갈림.

💡 GPU를 두 개로 나눈 이유는 당시(2012) GPU 메모리가 부족해서 모델을 한 장에 다 올릴 수 없었기 때문. 지금은 한 GPU에 올리는 버전(아래 코드)이 일반적

### 15. AlexNet 구현

전반적인 코드는 LeNet과 거의 동일 → 네트워크(모델 구조)가 어떻게 다른지 위주로 보면 됨

### 15-1. LeNet 대비 바뀐 점 / 같은 점

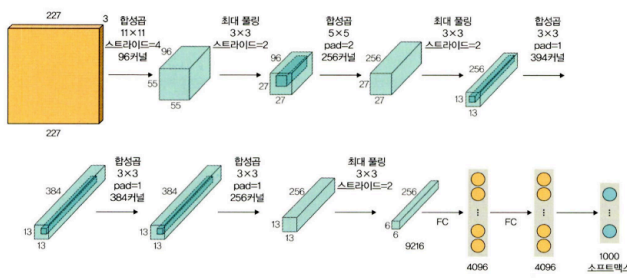
| 단계                                       | 코드          | LeNet과 비교                                          |
|------------------------------------------|-------------|----------------------------------------------------|
| 라이브러리 호출                                 | 6-22        | 동일 (matplotlib만 빠짐)                                |
| 전처리 <code>ImageTransform</code>          | 6-23        | 동일 (설명 생략)                                         |
| 데이터 분리                                   | 6-24        | 동일 — train 400 / val 92 / test 10                  |
| 커스텀 데이터셋<br><code>DogvsCatDataset</code> | 6-25        | 동일                                                 |
| 변수 값                                     | 6-26        | <b>size = 256</b> (LeNet은 224)                     |
| 데이터셋·데이터로더                               | 6-27, 6-28  | test용도 추가로 정의                                      |
| <b>모델 네트워크</b>                           | 6-29        | ★ <b>완전히 다름</b>                                    |
| 옵티마이저·손실 함수                              | 6-31        | 동일 — SGD(lr=0.001, momentum=0.9), CrossEntropyLoss |
| 학습 함수                                    | 6-33        | 거의 동일 (best_acc 저장 부분 빠짐)                          |
| 테스트·시각화                                  | 6-35 ~ 6-38 | 동일 (CSV 파일명만 alexnet.csv)                          |

### 15-2. 예제 네트워크

- 원래 AlexNet의 마지막 계층엔 뉴런 **1000개** → 예제에서는 클래스 **2개**(개/고양이)만 사용

| 순서 | 연산                       | 출력                    |
|----|--------------------------|-----------------------|
| 입력 | —                        | 227×227×3             |
| 1  | 합성곱 11×11, 스트라이드 4, 96커널 | 55×55×96              |
| 2  | 최대 풀링 3×3, 스트라이드 2       | 27×27×96              |
| 3  | 합성곱 5×5, pad 2, 256커널    | 27×27×256             |
| 4  | 최대 풀링 3×3, 스트라이드 2       | 13×13×256             |
| 5  | 합성곱 3×3, pad 1, 384커널    | 13×13×384             |
| 6  | 합성곱 3×3, pad 1, 384커널    | 13×13×384             |
| 7  | 합성곱 3×3, pad 1, 256커널    | 13×13×256             |
| 8  | 최대 풀링 3×3, 스트라이드 2       | 6×6×256 = <b>9216</b> |
| 9  | FC                       | 4096                  |
| 10 | FC                       | 4096                  |
| 11 | 소프트맥스                    | 1000                  |

♥ 그림 6-11 AlexNet 예제 네트워크



### 15-3. 데이터 준비에서 짚을 점

#### 데이터를 400개로 제한한 이유

- 노트북(혹은 서버, 코랩)의 성능(CPU/GPU, 메모리)이 좋지 않다고 가정
- AlexNet은 파라미터 **6,000만 개** → 충분한 데이터가 없으면 **과적합**이 발생해 테스트 성능이 나쁨
- 이 예제는 데이터셋을 많이 제한했으므로 **성능이 좋지 않다는 것을 미리 언급**
- 성능을 높이려면: 캐글 이미지 **전부** 사용 + 전처리에서 데이터를 많이 **확장**(`RandomRotation`, `RandomHorizontalFlip` 등 = 데이터 증강)

#### 커스텀 데이터셋 (코드 6-25)

- `torch.utils.data.Dataset` 을 상속받아 커스텀 데이터셋(custom dataset) 정의
- 주석 정리

| 코드                                         | 의미                                                  |
|--------------------------------------------|-----------------------------------------------------|
| <code>self.file_list</code>                | 이미지 데이터가 위치한 파일 경로                                  |
| <code>self.transform</code>                | 이미지 데이터 전처리                                         |
| <code>self.phase</code>                    | <code>ImageTransform()</code> 에서 정의한 'train'과 'val' |
| <code>self.file_list[idx]</code>           | 이미지 데이터의 인덱스 가져오기                                   |
| <code>split('/')[0].split('.')[0]</code>   | 레이블 값 가져오기 (dog → 1, cat → 0)                       |
| <code>return img_transformed, label</code> | 전처리가 적용된 이미지와 레이블 반환                                |

#### 변수 값 (코드 6-26)

| 변수         | 값                     | 비고                                                                                   |
|------------|-----------------------|--------------------------------------------------------------------------------------|
| size       | <b>256</b>            | AlexNet은 깊이가 깊은 네트워크 → 이미지 크기가 256이 아니면 풀링층 때<br>문에 크기가 계속 줄어들어 <b>오류</b> 가 발생할 수 있음 |
| mean       | (0.485, 0.456, 0.406) | ImageNet 평균                                                                          |
| std        | (0.229, 0.224, 0.225) | ImageNet 표준편차                                                                        |
| batch_size | 32                    |                                                                                      |

#### 데이터셋·데이터로더 (코드 6-27, 6-28)

- 훈련 데이터 1개 출력: `torch.Size([3, 256, 256])`, 레이블 `1` → (채널, 너비, 높이)

- 데이터로더 배치 출력: `torch.Size([32, 3, 256, 256])` + 레이블 32개
- `test_dataloader` 도 `shuffle=False` 로 추가 정의

#### 15-4. AlexNet 모델 네트워크 (코드 6-29) ★

- 사전 훈련된 네트워크와 유사하게 정의
- 구성: 합성곱(Conv2d) + 활성화 함수(ReLU) + 풀링(MaxPool2d) 반복 → 두 개의 완전연결층 + 출력층
- 구조를 3덩어리로 나눔

| 덩어리 | 이름                                                    | 역할                  |
|-----|-------------------------------------------------------|---------------------|
| ①   | <code>self.features</code> (nn.Sequential)            | 합성곱·ReLU·풀링으로 특징 추출 |
| ②   | <code>self.avgpool</code> (AdaptiveAvgPool2d((6, 6))) | 출력 크기를 6×6으로 고정     |
| ③   | <code>self.classifier</code> (nn.Sequential)          | Dropout·Linear로 분류  |

`forward` 흐름: features → avgpool → `torch.flatten(x, 1)` → classifier

💡 `nn.Sequential`: 층들을 순서대로 담는 상자. LeNet처럼 층마다 `self.cnn1 = ...` 로 이름 붙이고 forward에 하나씩 적을 필요 없이, 상자째로 한 번에 통과시킴.

💡 `torch.flatten(x, 1)`: LeNet의 `view(out.size(0), -1)` 과 같은 역할. 1번 차원부터 펴라 = 배치(0번)는 유지하고 나머지를 한 줄로.

##### ① features 상세

| #  | 층         | 설정                                     |
|----|-----------|----------------------------------------|
| 0  | Conv2d    | 3 → 64, kernel 11, stride 4, padding 2 |
| 1  | ReLU      | inplace=True                           |
| 2  | MaxPool2d | kernel 3, stride 2                     |
| 3  | Conv2d    | 64 → 192, kernel 5, padding 2          |
| 4  | ReLU      | inplace=True                           |
| 5  | MaxPool2d | kernel 3, stride 2                     |
| 6  | Conv2d    | 192 → 384, kernel 3, padding 1         |
| 7  | ReLU      | inplace=True                           |
| 8  | Conv2d    | 384 → 256, kernel 3, padding 1         |
| 9  | ReLU      | inplace=True                           |
| 10 | Conv2d    | 256 → 256, kernel 3, padding 1         |
| 11 | ReLU      | inplace=True                           |
| 12 | MaxPool2d | kernel 3, stride 2                     |

##### ③ classifier 상세

| # | 층       | 설정                    |
|---|---------|-----------------------|
| 0 | Dropout | p=0.5                 |
| 1 | Linear  | 256×6×6 = 9216 → 4096 |

| # | 층       | 설정                |
|---|---------|-------------------|
| 2 | ReLU    | inplace=True      |
| 3 | Dropout | p=0.5             |
| 4 | Linear  | 4096 → <b>512</b> |
| 5 | ReLU    | inplace=True      |
| 6 | Linear  | 512 → <b>2</b>    |

**Dropout:** 학습 중 뉴런을 무작위로(p=0.5 → 절반) 꺼서 특정 뉴런에 의존하지 않게 함 → **과적합 방지**. 파라미터가 몰린 완전연결층 앞에 배치

#### 15-5. ① `inplace=True`

- inplace 연산 = 연산 결과값을 **새로운 변수에 저장하지 않고 기존 데이터를 대체하는 것**
- 즉, 기존 값을 연산 결과값으로 덮어써서 **기존 값들을 무시하겠다는 의미** → 메모리 절약

#### 15-6. ② `nn.AdaptiveAvgPool2d`

|             | <code>nn.AvgPool2d</code> | <code>nn.AdaptiveAvgPool2d</code> |
|-------------|---------------------------|-----------------------------------|
| 지정하는 것      | 커널 크기, 스트라이드, 패딩          | 출력 크기만                            |
| 출력 크기       | 공식으로 계산되어 결정됨             | 내가 정한 대로 고정                       |
| 커널·스트라이드·패딩 | 내가 정함                     | 출력 크기에 맞게 <b>자동 계산</b>            |

**AvgPool2d 출력 크기 공식** — 입력 (N, C, H<sub>in</sub>, W<sub>in</sub>) → 출력 (N, C, H<sub>out</sub>, W<sub>out</sub>)

$$H_{out} = \lfloor (H_{in} + 2 \times padding[0] - kernel\_size[0]) / stride[0] + 1 \rfloor$$

$$W_{out} = \lfloor (W_{in} + 2 \times padding[1] - kernel\_size[1]) / stride[1] + 1 \rfloor$$

( $\lfloor \rfloor$  = 내림, [0] = 높이 방향 값, [1] = 너비 방향 값)

- 교재 예: `nn.AvgPool2d((3, 2), stride=(2, 1))` 로 5×5 텐서 → 3×3, 7×7 → 4×4로 줄이는 효과
- AdaptiveAvgPool2d는 H<sub>out</sub>, W<sub>out</sub>가 이미 정해져 있으니 위 공식을 거꾸로 풀어 커널·스트라이드·패딩을 구함 → **출력 크기 조정이 상당히 쉬워짐**
- 입력 크기에 변동이 있고, CNN 위쪽에 완전연결층을 쓰는 경우 유용

💡 AvgPool2d = "3칸씩 묶어서 평균 내" (결과 크기는 알아서), AdaptiveAvgPool2d = "결과를 무조건 6×6으로 만들어" (묶는 방법은 알아서). 입력이 224든 256이든 항상 6×6 → 바로 뒤 `Linear(9216, 4096)` 가 절대 안 깨짐.

#### 15-7. 모델 구조 확인 (코드 6-30, 6-32)

- `model = AlexNet()` → `model.to(device)`
- `summary(model, input_size=(3, 256, 256))`

| Layer (type) | Output Shape     | Param # |
|--------------|------------------|---------|
| Conv2d-1     | [-1, 64, 63, 63] | 23,296  |
| ReLU-2       | [-1, 64, 63, 63] | 0       |

| Layer (type)         | Output Shape      | Param #    |
|----------------------|-------------------|------------|
| MaxPool2d-3          | [-1, 64, 31, 31]  | 0          |
| Conv2d-4             | [-1, 192, 31, 31] | 307,392    |
| ReLU-5               | [-1, 192, 31, 31] | 0          |
| MaxPool2d-6          | [-1, 192, 15, 15] | 0          |
| Conv2d-7             | [-1, 384, 15, 15] | 663,936    |
| ReLU-8               | [-1, 384, 15, 15] | 0          |
| Conv2d-9             | [-1, 256, 15, 15] | 884,992    |
| ReLU-10              | [-1, 256, 15, 15] | 0          |
| Conv2d-11            | [-1, 256, 15, 15] | 590,080    |
| ReLU-12              | [-1, 256, 15, 15] | 0          |
| MaxPool2d-13         | [-1, 256, 7, 7]   | 0          |
| AdaptiveAvgPool2d-14 | [-1, 256, 6, 6]   | 0          |
| Dropout-15           | [-1, 9216]        | 0          |
| Linear-16            | [-1, 4096]        | 37,752,832 |
| ReLU-17              | [-1, 4096]        | 0          |
| Dropout-18           | [-1, 4096]        | 0          |
| Linear-19            | [-1, 512]         | 2,097,664  |
| ReLU-20              | [-1, 512]         | 0          |
| Linear-21            | [-1, 2]           | 1,026      |

| 항목                              | 값          |
|---------------------------------|------------|
| Total params                    | 42,321,218 |
| Trainable params                | 42,321,218 |
| Non-trainable params            | 0          |
| Input size (MB)                 | 0.75       |
| Forward/backward pass size (MB) | 10.90      |
| Params size (MB)                | 161.44     |
| Estimated Total Size (MB)       | 173.10     |

### 크기·파라미터 직접 계산

| 층           | 출력 크기 계산                                             | 파라미터 계산                                              |
|-------------|------------------------------------------------------|------------------------------------------------------|
| Conv2d-1    | $\lfloor (256 + 4 - 11)/4 \rfloor + 1 = \mathbf{63}$ | $(3 \times 11 \times 11) \times 64 + 64 = 23,296$    |
| MaxPool2d-3 | $\lfloor (63 - 3)/2 \rfloor + 1 = \mathbf{31}$       | 0                                                    |
| Conv2d-4    | $(31 + 4 - 5)/1 + 1 = \mathbf{31}$                   | $(64 \times 5 \times 5) \times 192 + 192 = 307,392$  |
| MaxPool2d-6 | $\lfloor (31 - 3)/2 \rfloor + 1 = \mathbf{15}$       | 0                                                    |
| Conv2d-7    | $(15 + 2 - 3)/1 + 1 = \mathbf{15}$                   | $(192 \times 3 \times 3) \times 384 + 384 = 663,936$ |
| Conv2d-9    | 15                                                   | $(384 \times 3 \times 3) \times 256 + 256 = 884,992$ |
| Conv2d-11   | 15                                                   | $(256 \times 3 \times 3) \times 256 + 256 = 590,080$ |

| 층                    | 출력 크기 계산                             | 파라미터 계산                                |
|----------------------|--------------------------------------|----------------------------------------|
| MaxPool2d-13         | $\lfloor (15 - 3)/2 \rfloor + 1 = 7$ | 0                                      |
| AdaptiveAvgPool2d-14 | 7 → 6 (강제 고정)                        | 0                                      |
| Linear-16            | 9216 → 4096                          | $9216 \times 4096 + 4096 = 37,752,832$ |
| Linear-19            | 4096 → 512                           | $4096 \times 512 + 512 = 2,097,664$    |
| Linear-21            | 512 → 2                              | $512 \times 2 + 2 = 1,026$             |

#### 💡 포인트

- padding=1 + kernel 3 + stride 1 조합 → **크기가 그대로 유지** (15 → 15). 깊게 쌓아도 크기가 안 줄게 하는 표준 설정
- 이번에도 파라미터의 약 **89%가 Linear-16(첫 FC)**에 몰림. 합성곱층 5개를 다 합쳐도 약 247만 개뿐
- LeNet(약 4,604만)보다 층은 훨씬 깊는데 파라미터는 오히려 적음(약 4,232만) → 풀링을 더 많이 해서 FC에 들어가는 크기가 작아졌기 때문 (89,888 vs 9,216)

### 15-8. 옵티마이저·손실 함수·학습 (코드 6-31, 6-33, 6-34)

- `optim.SGD(model.parameters(), lr=0.001, momentum=0.9)`, `nn.CrossEntropyLoss()` → LeNet과 동일
- `train_model` → LeNet과 같은 코드라 설명 생략 (단, best\_acc 갱신·출력 부분이 빠짐)
- 에포크 10 → 예제 진행 PC 성능과 학습 시간을 고려한 **최소한의 에포크**

#### 학습 결과

| Epoch | train Loss | train Acc | val Loss | val Acc |
|-------|------------|-----------|----------|---------|
| 1/10  | 0.6929     | 0.4950    | 0.6926   | 0.5109  |
| ...   | ...        | ...       | ...      | ...     |
| 9/10  | 0.6925     | 0.5025    | 0.6924   | 0.5109  |
| 10/10 | 0.6924     | 0.5025    | 0.6924   | 0.5109  |

- train 배치당 약 2.15~2.40초 (LeNet 1.4~1.6초보다 느림), `Training complete in 6m 9s`
- 훈련·검증 데이터셋이 적어 **학습 결과가 좋지 않음** → PC 성능이 좋다면 데이터셋을 늘려 학습해 볼 것

#### 💡 결과

- Loss가 처음부터 끝까지 **0.693 근처** →  $0.693 = \ln 2$  = 두 클래스를 **50:50으로 찍을 때의 크로스엔트로피 값**. 즉 **거의 학습이 안 됨**
- val Acc가 10 epoch 내내 **0.5109로 고정** → 92장 중 47장. 모델이 **한 클래스로만** 계속 예측하고 있다는 신호
- 원인: 데이터 400장 + 깊은 네트워크를 **처음부터(사전학습 없이)** 학습 + lr 0.001로 작음 + 10 epoch. 깊은 모델일수록 데이터·학습량이 훨씬 더 필요함

### 15-9. 테스트 & 결과 (코드 6-35 ~ 6-38)

- 테스트 코드는 LeNet과 동일: `torch.no_grad()` → val 전처리 → `unsqueeze(0)` → softmax 후 개별 확률 → id·확률을 DataFrame에 저장 → `alexnet.csv` 로 저장

- `_id = test_path.split('/')[1].split('.')[1]` → 이미지 번호 가져오기 (예: `dog.113.jpg` → `113`)
- 실행 결과 `10/10 [00:00<00:00, 16.72it/s]` → 처리 완료 의미

#### res.head(10) 결과

|   | id  | label    |
|---|-----|----------|
| 0 | 207 | 0.493234 |
| 1 | 99  | 0.494147 |
| 2 | 116 | 0.493472 |
| 3 | 42  | 0.493139 |
| 4 | 162 | 0.493536 |
| 5 | 194 | 0.493116 |
| 6 | 115 | 0.493754 |
| 7 | 186 | 0.493568 |
| 8 | 39  | 0.492973 |
| 9 | 190 | 0.493639 |

- `label > 0.5` → 개, `< 0.5` → 고양이
- 10개 전부 **0.493 근처** → **전부 고양이**로 예측 (이번엔 sort를 안 해서 id가 섞인 순서로 나옴)

#### 시각화 (그림 6-12): `display_image_grid(test_images_filepaths)` → 10장 모두 제목이 **cat**

- 개 사진에도 전부 cat → 앞에서 본 "val Acc 0.5109 고정 = 한 클래스로만 예측"과 같은 현상
- (LeNet 때와 같은 `random.choice` 함정도 그대로 있음)

▼ 그림 6-12 테스트 데이터셋에 대한 AlexNet 모델의 예측 결과



## 15-10. 결론

- 예측 결과가 좋지 않음 → 데이터셋이 많으면 성능이 좋아질 수 있음
- 책의 예제들은 실습할 PC(노트북) 성능을 고려해 데이터셋을 제한함
- 성능은 **파라미터 튜닝**으로도 향상시킬 수 있음 → 자세한 내용은 **8장 '성능 최적화'**

#### LeNet vs AlexNet 실습 비교

|                  | LeNet (예제)         | AlexNet (예제)                    |
|------------------|--------------------|---------------------------------|
| 입력 크기            | 3×224×224          | 3×256×256                       |
| 합성곱층             | 2개 (16, 32채널)      | 5개 (64 → 192 → 384 → 256 → 256) |
| 풀링               | MaxPool 2번         | MaxPool 3번 + AdaptiveAvgPool    |
| FC 입력 크기         | 89,888             | 9,216                           |
| 완전연결층            | 512 → 2            | 4096 → 512 → 2                  |
| 과적합 방지           | 없음                 | Dropout(0.5) × 2                |
| 마지막 Softmax      | 있음 (손실 함수와 중복)     | 없음 (정석)                         |
| 총 파라미터           | 46,038,242         | 42,321,218                      |
| 학습 시간 (10 epoch) | 4m 6s              | 6m 9s                           |
| 최종 val Acc       | 0.6304 (최고 0.6413) | 0.5109                          |
| 테스트 예측           | cat·dog 섞임         | 전부 cat                          |

💡 모델이 크고 깊다고 무조건 좋은 게 아님. **데이터 양에 맞는 모델 크기**가 중요하고, 데이터가 적을 땐 사전 학습 모델(전이 학습)이나 데이터 증강이 필요

## 6.1.3 VGGNet

### 16-1. VGGNet이란?

| 항목    | 내용                                                                              |
|-------|---------------------------------------------------------------------------------|
| 발표    | 카렌 시모니안(Karen Simonyan), 앤드류 지서만(Andrew Zisserman)                              |
| 논문    | 2015 ICLR, "Very deep convolutional networks for large-scale image recognition" |
| 탄생 목적 | 합성곱층의 <b>파라미터 수를 줄이고 훈련 시간을 개선</b>                                              |
| 핵심 질문 | 네트워크를 <b>깊게</b> 만드는 것이 성능에 어떤 영향을 미치는가?                                         |
| 실험 설계 | <b>깊이의 영향만</b> 보기 위해 합성곱층 필터/커널 크기를 가장 작은 <b>3×3</b> 으로 고정                      |
| 종류    | 네트워크 계층의 총 개수에 따라 <b>VGG11, VGG13, VGG16, VGG19</b> 등                           |

💡 실험에서 "변수 하나만 바꾸기"와 같음. 커널 크기 같은 다른 조건을 다 고정해 두고 **층 수(깊이)**만 늘려 보면서, 깊이가 성능을 올리는지 확인한 것.

#### 💡 왜 하필 3×3?

- 3×3을 **2번** 쌓으면 5×5 하나와 같은 범위(수용 영역)를 봄, **3번** 쌓으면 7×7과 같음
- 그런데 파라미터는 더 적음: 채널 C 기준 5×5 하나 =  $25C^2$ , 3×3 두 개 =  $2 \times 9C^2 = 18C^2$
- 층마다 ReLU가 붙으니 **비선형성도 더 많아짐** → 더 복잡한 패턴 표현 가능

### 16-2. VGG16 구조

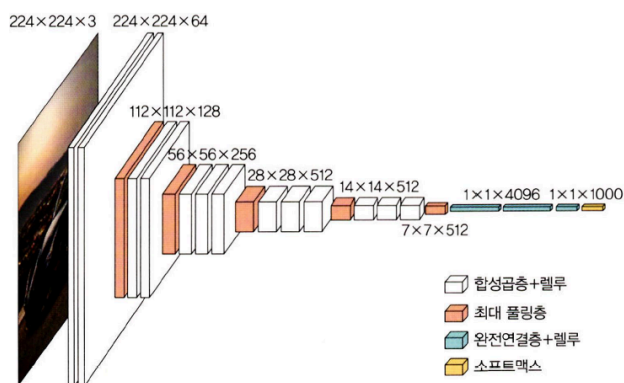
224×224×3 → 224×224×64 → 112×112×128 → 56×56×256 → 28×28×512 → 14×14×512 → 7×7×512 → 1×1×4096 → 1×1×4096 → 1×1×1000

| 범례    | 의미           |
|-------|--------------|
| 흰 블록  | 합성곱층 + ReLU  |
| 주황 블록 | 최대 풀링층       |
| 파란 블록 | 완전연결층 + ReLU |
| 노란 블록 | 소프트맥스        |

| 특징       | 내용                                     |
|----------|----------------------------------------|
| 파라미터     | 총 1억 3,300만 개                          |
| 합성곱 커널   | 모두 3×3                                 |
| 최대 풀링    | 커널 2×2, 스트라이드 2                        |
| 첫 합성곱 결과 | 64개의 224×224 특성 맵 (224×224×64)         |
| 활성화 함수   | 마지막 16번째 계층을 제외하고 모두 ReLU (마지막은 소프트맥스) |

💡 패턴: 풀링할 때마다 크기는 절반, 채널은 두 배(512에서 멈춤) → 224 → 112 → 56 → 28 → 14 → 7

▼ 그림 6-13 VGG16 구조



16-3. 표 6-3 VGG16 구조 상세

| 계층 유형  | 특성 맵 | 크기      | 커널 크기 | 스트라이드 | 활성화 함수 |
|--------|------|---------|-------|-------|--------|
| 이미지    | 1    | 224×224 | –     | –     | –      |
| 합성곱층   | 64   | 224×224 | 3×3   | 1     | ReLU   |
| 합성곱층   | 64   | 224×224 | 3×3   | 1     | ReLU   |
| 최대 풀링층 | 64   | 112×112 | 2×2   | 2     | –      |
| 합성곱층   | 128  | 112×112 | 3×3   | 1     | ReLU   |
| 합성곱층   | 128  | 112×112 | 3×3   | 1     | ReLU   |
| 최대 풀링층 | 128  | 56×56   | 2×2   | 2     | –      |
| 합성곱층   | 256  | 56×56   | 3×3   | 1     | ReLU   |
| 합성곱층   | 256  | 56×56   | 3×3   | 1     | ReLU   |
| 합성곱층   | 256  | 56×56   | 3×3   | 1     | ReLU   |
| 합성곱층   | 256  | 56×56   | 3×3   | 1     | ReLU   |

| 계층 유형  | 특성 맵 | 크기    | 커널 크기 | 스트라이드 | 활성화 함수 |
|--------|------|-------|-------|-------|--------|
| 최대 풀링층 | 256  | 28×28 | 2×2   | 2     | –      |
| 합성곱층   | 512  | 28×28 | 3×3   | 1     | ReLU   |
| 합성곱층   | 512  | 28×28 | 3×3   | 1     | ReLU   |
| 합성곱층   | 512  | 28×28 | 3×3   | 1     | ReLU   |
| 합성곱층   | 512  | 28×28 | 3×3   | 1     | ReLU   |
| 최대 풀링층 | 512  | 14×14 | 2×2   | 2     | –      |
| 합성곱층   | 512  | 14×14 | 3×3   | 1     | ReLU   |
| 합성곱층   | 512  | 14×14 | 3×3   | 1     | ReLU   |
| 합성곱층   | 512  | 14×14 | 3×3   | 1     | ReLU   |
| 합성곱층   | 512  | 14×14 | 3×3   | 1     | ReLU   |
| 최대 풀링층 | 512  | 7×7   | 2×2   | 2     | –      |
| 완전연결층  | –    | 4096  | –     | –     | ReLU   |
| 완전연결층  | –    | 4096  | –     | –     | ReLU   |
| 완전연결층  | –    | 1000  | –     | –     | 소프트맥스  |

## 17. VGG11 구현

- 예제는 VGGNet 중 **가장 간단한 VGG11**을 파이토치로 구현
- VGG16이나 VGG19를 구현하고 싶다면 VGG11에서 **더 깊게 쌓아 올리면** 됨

### 17-1. 라이브러리 (코드 6-39)

- 새로 등장: `import copy` (① 객체 복사), `torch.utils.data as data`, `torchvision.datasets as Datasets`
- `device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')`

### 17-2. ① 객체 복사: 단순 복사 / 얕은 복사 / 깊은 복사

객체 복사는 크게 **얕은 복사(shallow copy)** 와 **깊은 복사(deep copy)** 로 나뉨

#### (1) 단순 복사 — `copy_o = original`

| 코드                                                                 | copy_o     | original            |
|--------------------------------------------------------------------|------------|---------------------|
| <code>original = [1, 2, 3]</code> , <code>copy_o = original</code> | [1, 2, 3]  | [1, 2, 3]           |
| <code>copy_o[2] = 10</code>                                        | [1, 2, 10] | [1, 2, 10] ← 같이 바뀜! |

#### (2) 얕은 복사 — `copy.copy()`

| 코드                                                                                       | 복사본            | original                |
|------------------------------------------------------------------------------------------|----------------|-------------------------|
| <code>original = [[1, 2], 3]</code> , <code>copy_o = copy.copy(original)</code>          | [[1, 2], 3]    | [[1, 2], 3]             |
| <code>copy_o[0] = 100</code> (바깥 요소 교체)                                                  | [100, 3]       | [[1, 2], 3] ← 안 바뀜      |
| <code>append = copy.copy(original)</code> , <code>append[0].append(4)</code> (안쪽 리스트 수정) | [[1, 2, 4], 3] | [[1, 2, 4], 3] ← 같이 바뀜! |

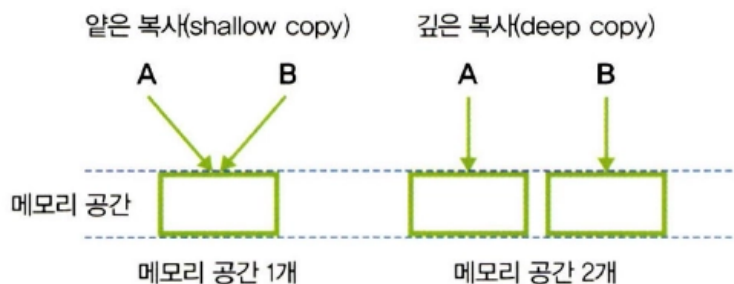
### (3) 깊은 복사 — `copy.deepcopy()`

| 코드                                                                 | 복사본            | original           |
|--------------------------------------------------------------------|----------------|--------------------|
| <code>copy_o = copy.deepcopy(original)</code>                      | [[1, 2], 3]    | [[1, 2], 3]        |
| <code>copy_o[0] = 100</code>                                       | [100, 3]       | [[1, 2], 3] ← 안 바뀜 |
| <code>append = copy.deepcopy(original), append[0].append(4)</code> | [[1, 2, 4], 3] | [[1, 2], 3] ← 안 바뀜 |

그림 6-14 얕은 복사와 깊은 복사

|         | 얕은 복사                   | 깊은 복사              |
|---------|-------------------------|--------------------|
| 메모리     | A, B가 메모리 공간 1개를 같이 가리킴 | A, B가 각자 메모리 공간 2개 |
| 한쪽 수정 시 | 다른 쪽도 영향                | 서로 독립              |

▼ 그림 6-14 얕은 복사와 깊은 복사



#### 💡 직관 (집 비유)

- **단순 복사** = 집 주소만 하나 더 적어 둔 것 → 같은 집. 누가 들어가서 가구를 바꾸면 둘 다 바뀜
- **얕은 복사** = 집 겉 건물만 새로 지었는데, 안의 가구(안쪽 리스트)는 원래 집 것을 같이 씀 → 방 배치(바깥 요소)를 바꾸는 건 각자지만, 공유 가구를 고치면 둘 다 바뀜
- **깊은 복사** = 가구까지 전부 새로 만든 완전히 다른 집

💡 17-8(코드 6-50)에서 검증 데이터에 다른 전처리를 줄 때 `deepcopy` 를 쓸 예정

### 17-3. VGG 모델 클래스 (코드 6-40)

| 부분                                                  | 내용                                                                                             |
|-----------------------------------------------------|------------------------------------------------------------------------------------------------|
| <code>self.features = features</code>               | VGG 모델에 대한 매개변수에서 받아 온 features 값(합성곱 계층 묶음)을 넣어 줌 → 설정(config)에 따라 VGG11/13/16/19를 바꿔 끼울 수 있음 |
| <code>self.avgpool = nn.AdaptiveAvgPool2d(7)</code> | 출력을 7×7로 고정                                                                                    |
| <code>self.classifier</code>                        | 완전연결층과 출력층 정의                                                                                  |

#### classifier 구성

| # | 층      | 설정                     |
|---|--------|------------------------|
| 0 | Linear | 512×7×7 = 25088 → 4096 |

| # | 층       | 설정                |
|---|---------|-------------------|
| 1 | ReLU    | inplace=True      |
| 2 | Dropout | 0.5               |
| 3 | Linear  | 4096 → 4096       |
| 4 | ReLU    | inplace=True      |
| 5 | Dropout | 0.5               |
| 6 | Linear  | 4096 → output_dim |

**forward:** features → avgpool → `h = x.view(x.shape[0], -1)` (펼치기) → classifier → `return x, h`

💡 출력을 두 개(예측값 x, 펼친 특징 h) 반환 → 그래서 뒤의 학습 코드에서 `y_pred, _ = model(x)` 처럼 두 번째 값은 `_`로 버림. h는 나중에 특징 벡터를 시각화·분석할 때 쓰려고 꺼내 둔 것

#### 17-4. 모델 유형 정의 (코드 6-41)

- 숫자 = output channel(출력 채널) → Conv2d를 수행하라는 의미. 출력 채널이 다음 계층의 입력 채널(input channel)이 됨
- 'M' = 최대 풀링(max pooling)을 수행하라는 의미

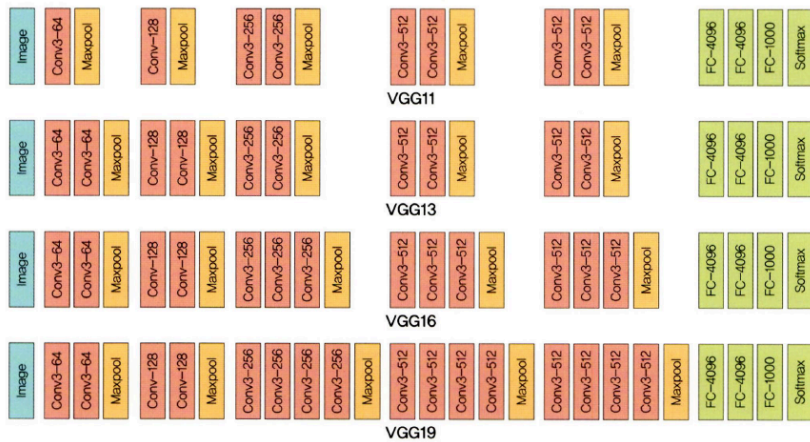
| 모델    | config                                                                                   | 구성                        |
|-------|------------------------------------------------------------------------------------------|---------------------------|
| VGG11 | [64, M, 128, M, 256, 256, M, 512, 512, M, 512, 512, M]                                   | 합성곱 8 + FC 3 = <b>11</b>  |
| VGG13 | [64, 64, M, 128, 128, M, 256, 256, M, 512, 512, M, 512, 512, M]                          | 합성곱 10 + FC 3 = <b>13</b> |
| VGG16 | [64, 64, M, 128, 128, M, 256, 256, 256, M, 512, 512, 512, M, 512, 512, M]                | 합성곱 13 + FC 3 = <b>16</b> |
| VGG19 | [64, 64, M, 128, 128, M, 256, 256, 256, 256, M, 512, 512, 512, 512, M, 512, 512, 512, M] | 합성곱 16 + FC 3 = <b>19</b> |

#### 그림 6-15 VGG의 다양한 모델 네트워크

- 4개 모델 모두: Image → [합성곱 블록 + Maxpool] × 5 → FC-4096 → FC-4096 → FC-1000 → Softmax
- 차이는 각 블록 안의 합성곱(Conv3) 개수뿐 (VGG11은 앞 두 블록에 1개씩, VGG19는 뒤 세 블록에 4개씩)

💡 **설정 리스트(config) 방식의 장점:** 레고 설명서처럼 리스트만 바꾸면 모델이 바뀐. 직접 `My_Vgg = [64, 64, 64, 'M', 128, 128, 128, 'M', 256, 256, 256, 'M']` 같은 새 구조도 만들 수 있음

▼ 그림 6-15 VGG의 다양한 모델에 대한 네트워크



## 17-5. VGG 계층 정의 함수 (코드 6-42 `get_vgg_layers`)

### 동작 흐름

1. `layers = []`, `in_channels = 3` (컬러 이미지로 시작)
2. config 값을 하나씩 꺼냄 (`for c in config`)
3. ① `assert c == 'M' or isinstance(c, int)` → 값이 올바른지 검사
4. 'M' 이면 → `nn.MaxPool2d(kernel_size=2)` 추가
5. 숫자이면 → `nn.Conv2d(in_channels, c, kernel_size=3, padding=1)` 생성
  - `batch_norm=True` → [conv2d, **BatchNorm2d(c)**, ReLU] 추가 (배치 정규화 + ReLU)
  - `batch_norm=False` → [conv2d, ReLU] 추가 (ReLU만)
  - `in_channels = c` (다음 층 입력 채널 갱신)
6. `return nn.Sequential(*layers)` → 네트워크의 모든 계층을 반환

### ① `assert` & `isinstance`

| 기호 | 함수                      | 의미                                    | 예시                                                                                                                                              |
|----|-------------------------|---------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------|
| Ⓐ  | <code>assert</code>     | 가정 설정문. 뒤의 조건이 True가 아니면 <b>에러 발생</b> | <code>a = 1</code> → <code>assert a == 11</code> → <b>AssertionError</b>                                                                        |
| Ⓑ  | <code>isinstance</code> | 주어진 조건이 True인지 판단 (자료형 확인)            | <code>isinstance(1, int)</code> → True /<br><code>isinstance(1.2, int)</code> → False /<br><code>isinstance('deep learning', str)</code> → True |

- 따라서 `assert c == 'M' or isinstance(c, int)` = c가 'M'도 아니고 int도 아니라면 오류 발생

## 17-6. 모델 계층 생성 (코드 6-43, 6-44)

- `vgg11_layers = get_vgg_layers(vgg11_config, batch_norm=True)` → 배치 정규화(batch normalization) 계층도 추가

### ① 배치 정규화 (`batch_norm`)

- 데이터의 **평균을 0, 표준편차를 1**로 분포시키는 것
- 각 계층에서 입력 데이터의 분포는 **앞 계층에서 업데이트된 가중치에 따라 변함**
- 계층마다 변하는 분포는 **학습 속도를 늦출 뿐 아니라 학습도 어렵게 함**
- → 각 계층의 입력에 대한 분산을 평균 0, 표준편차 1로 분포시키는 것이 batch\_norm (자세한 건 **8장**)

#### VGG11 계층 출력 (print(vgg11\_layers)) — 0~28번

| 블록 | 계층 번호 | 구성                                                                      |
|----|-------|-------------------------------------------------------------------------|
| 1  | 0-3   | Conv2d(3 → 64) · BN(64) · ReLU · MaxPool                                |
| 2  | 4-7   | Conv2d(64 → 128) · BN(128) · ReLU · MaxPool                             |
| 3  | 8-14  | Conv2d(128 → 256) · BN · ReLU · Conv2d(256 → 256) · BN · ReLU · MaxPool |
| 4  | 15-21 | Conv2d(256 → 512) · BN · ReLU · Conv2d(512 → 512) · BN · ReLU · MaxPool |
| 5  | 22-28 | Conv2d(512 → 512) · BN · ReLU · Conv2d(512 → 512) · BN · ReLU · MaxPool |

- 모든 Conv2d: kernel\_size=(3, 3), stride=(1, 1), padding=(1, 1) → **크기 유지**
- 모든 MaxPool2d: kernel\_size=2, stride=2 → **크기 절반**
- BatchNorm2d 기본 설정: eps=1e-05, momentum=0.1, affine=True, track\_running\_stats=True
- 확인 결과 **배치 정규화(BatchNorm2d)**가 추가된 것을 볼 수 있음

#### 17-7. VGG11 전체 네트워크 (코드 6-45)

- `OUTPUT_DIM = 2` (개와 고양이 두 개의 클래스) → `model = VGG(vgg11_layers, OUTPUT_DIM)`
- 결과: features(위 0~28) + `AdaptiveAvgPool2d(output_size=7)` + classifier

| # | classifier 층                                 |
|---|----------------------------------------------|
| 0 | Linear(in_features=25088, out_features=4096) |
| 1 | ReLU(inplace=True)                           |
| 2 | Dropout(p=0.5)                               |
| 3 | Linear(4096 → 4096)                          |
| 4 | ReLU(inplace=True)                           |
| 5 | Dropout(p=0.5)                               |
| 6 | Linear(4096 → 2)                             |

💡 이번에도 약 **93%가 완전연결층**(특히 첫 FC 하나가 약 80%). 합성곱 8개 합쳐도 약 922만 개 → "깊이를 늘려도 3×3이라 합성곱 파라미터는 적다"는 VGG 아이디어 확인

#### 17-8. 사전 훈련된 모델 사용 (코드 6-46)

- VGG 모델은 **사전 훈련된 모델**: 이미 누군가가 **대용량 이미지 데이터로 학습**시켰고, 최상의 상태로 튜닝을 거쳐 모든 사람이 사용할 수 있도록 공유한 모델

- 코드 한두 줄로 쉽게 가져다 사용 가능

```
pretrained_model = models.vgg11_bn(pretrained=True)
```

| 기호 | 부분                           | 의미                                |
|----|------------------------------|-----------------------------------|
| ㉠  | <code>vgg11_bn</code>        | VGG11 기본 모델에 배치 정규화가 적용된 모델 사용    |
| ㉡  | <code>pretrained=True</code> | 사전 훈련된 모델 사용 (미리 학습된 파라미터 값들을 사용) |

- 출력 구조는 직접 만든 VGG11과 동일, 단 avgpool = `AdaptiveAvgPool2d(output_size=(7, 7))`, 마지막 Linear = **4096** → **1000** (ImageNet 1000개 클래스)

📝 최신 torchvision에서는 `pretrained=True` 대신 `weights=models.VGG11_BN_Weights.DEFAULT` 사용 (pretrained는 경고가 뜨거나 제거됨)

- 앞에서 수십 줄 넘게 구현한 코드가 의미 없게 느껴질 수 있지만 → 향후 연습·연구에서는 사전 훈련된 모델을 호출해 쓰게 되고, 책에서는 네트워크를 어떻게 구현하고 사용하는지 보여 주려고 직접 정의한 것. 이렇게 이해해 두면 **My\_Vgg** 같은 새 config도 정의해서 쓸 수 있음

## 17-9. 이미지 전처리 (코드 6-47)

| 순서 | train_transforms                           | test_transforms                            |
|----|--------------------------------------------|--------------------------------------------|
| 1  | <code>Resize((256, 256))</code>            | <code>Resize((256, 256))</code>            |
| 2  | <code>RandomRotation(5)</code>             | –                                          |
| 3  | <code>RandomHorizontalFlip(0.5)</code>     | –                                          |
| 4  | <code>ToTensor()</code>                    | <code>ToTensor()</code>                    |
| 5  | <code>Normalize(ImageNet mean, std)</code> | <code>Normalize(ImageNet mean, std)</code> |

### ① 새로 등장한 파라미터

| 기호 | 함수                                         | 의미                                  |
|----|--------------------------------------------|-------------------------------------|
| ㉠  | <code>transforms.Resize((256, 256))</code> | 이미지를 주어진 크기로 재조정 → <b>256×256</b>   |
| ㉡  | <code>transforms.RandomRotation(5)</code>  | <b>5도 이하</b> 로 이미지를 무작위 회전 (데이터 증강) |

💡 `Resize((256, 256))` 처럼 튜플이면 가로·세로를 정확히 맞춤(비율이 찌그러질 수 있음), `Resize(256)` 처럼 숫자 하나면 짧은 변을 256에 맞추고 비율 유지

## 17-10. ImageFolder로 데이터셋 불러오기 (코드 6-48)

- `train_path = '../chap06/data/catanddog/train'`, `test_path = '../chap06/data/catanddog/test'`
- `torchvision.datasets.ImageFolder(경로, transform=...)`

### ① ImageFolder는 언제 쓰나? (파라미터는 5장에서 다룸)

- 계층적인 폴더 구조를 가진 데이터셋을 불러올 때 사용
- 그림 6-16: `train` 폴더 하위에 **Cat**, **Dog** 폴더가 위치

```
catanddog/
├─ train/
│  └─ Cat/
```

```
|   └─ Dog/
└─ test/
```

| 비교   | 앞 예제 (LeNet-AlexNet)           | VGG 예제                   |
|------|--------------------------------|--------------------------|
| 데이터셋 | 커스텀 <code>Dataset</code> 직접 정의 | <code>ImageFolder</code> |
| 레이블  | 파일명에서 split                    | 폴더명에서 자동                 |

- 결과: 훈련 529개, 테스트 12개 (출력의 `(None, None)` 은 `print` 안에 `print` 를 넣어서 생긴 것, 의미 없음)

### 17-11. 훈련/검증 분할 (코드 6-49)

| 변수                            | 계산                                                                  | 값          |
|-------------------------------|---------------------------------------------------------------------|------------|
| <code>VALID_RATIO</code>      |                                                                     | 0.9        |
| <code>n_train_examples</code> | $\text{int}(529 \times 0.9) \rightarrow$ 전체 훈련 데이터 중 90%를 훈련 데이터셋으로 | <b>476</b> |
| <code>n_valid_examples</code> | $529 - 476 \rightarrow$ 전체 훈련 데이터 중 10%를 검증 데이터셋으로                  | <b>53</b>  |

① `data.random_split(train_dataset, [n_train_examples, n_valid_examples])`

- 훈련과 검증 데이터셋을 나누는 용도
- ㉠ 첫 번째 파라미터: 분할에 사용될 데이터셋
- ㉡ 두 번째 파라미터: 훈련과 검증 데이터셋의 크기 지정 [훈련 크기, 검증 크기]
- ⚠ 데이터가 데이터로더로 넘어간 이후에는 분리가 불가능 → 반드시 데이터셋(dataset) 단계에서 진행

📖 교재는 "파이썬의 `random_split()`"이라 했지만 정확히는 파이토치(`torch.utils.data`) 함수

### 17-12. 검증 데이터 전처리 (코드 6-50) — deepcopy가 쓰이는 곳!

- `valid_data = copy.deepcopy(valid_data)`
- `valid_data.dataset.transform = test_transforms`
- 훈련 데이터셋에서 분리된 `valid_data`를 복사한 뒤, `test_transforms`로 전처리 적용 (검증은 증강 없이)

💡 왜 deepcopy가 꼭 필요할까?

- `random_split` 은 데이터를 실제로 둘로 쪼개는 게 아니라, 같은 원본 데이터셋을 공유하면서 "몇 번째 인덱스를 쓸지"만 나눔
- 그 상태에서 `valid_data.dataset.transform` 을 바꾸면 → 공유 중인 원본의 transform이 바뀌어 **train\_data까지 증강이 꺼져 버림** (얕은 복사의 함정!)
- deepcopy로 원본까지 통째로 복제해 두면 → 검증 쪽만 `test_transforms`, 훈련 쪽은 `train_transforms` 유지

### 17-13. 데이터 수 확인 & 데이터로더 (코드 6-51, 6-52)

| 데이터                           | 개수         |
|-------------------------------|------------|
| Number of training examples   | <b>476</b> |
| Number of validation examples | <b>53</b>  |

|                            |           |
|----------------------------|-----------|
| 데이터                        | 개수        |
| Number of testing examples | <b>12</b> |

| 이터레이터                       | 데이터                       | 설정                                                                        |
|-----------------------------|---------------------------|---------------------------------------------------------------------------|
| <code>train_iterator</code> | <code>train_data</code>   | <code>shuffle=True</code> (훈련 데이터셋은 임의로 섞어서), <code>batch_size=128</code> |
| <code>valid_iterator</code> | <code>valid_data</code>   | <code>batch_size=128</code>                                               |
| <code>test_iterator</code>  | <code>test_dataset</code> | <code>batch_size=128</code>                                               |

- `BATCH_SIZE = 128` (앞 예제의 32보다 큼)

#### 17-14. 옵티마이저·손실 함수 (코드 6-53)

- `optimizer = optim.Adam(model.parameters(), lr=1e-7)` ← 앞 예제와 달리 **Adam**
- `criterion = nn.CrossEntropyLoss()`
- `model.to(device)`, `criterion.to(device)`

⚠ **lr = 1e-7 = 0.0000001** → 매우 작음 (Adam의 일반적인 기본값은 1e-3). 가중치가 거의 안 움직여서 학습이 사실상 진행되지 않음 → 아래 학습 결과가 나쁜 핵심 원인 중 하나

#### 17-15. 정확도 측정 함수 (코드 6-54 `calculate_accuracy`)

| 단계 | 코드                                                              | 의미                                                   |
|----|-----------------------------------------------------------------|------------------------------------------------------|
| 1  | <code>top_pred = y_pred.argmax(1, keepdim=True)</code>          | 클래스 방향에서 가장 큰 값의 <b>인덱스</b> = 예측 클래스 (형태 유지: (N, 1)) |
| 2  | ① <code>correct = top_pred.eq(y.view_as(top_pred)).sum()</code> | 예측이 정답과 일치하는 경우 그 <b>개수의 합</b>                       |
| 3  | <code>acc = correct.float() / y.shape[0]</code>                 | 맞힌 개수 ÷ 배치 크기 = 정확도                                  |

##### ① 상세

| 기호 | 부분                          | 의미                                                                                                                                                           |
|----|-----------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Ⓐ  | <code>eq</code>             | <b>equal</b> 의 약자, 서로 같은지 비교 (같으면 True)                                                                                                                      |
| Ⓑ  | <code>view_as(other)</code> | <code>other</code> 의 텐서 크기를 사용 = <code>view(other.size())</code> 와 같음 → <code>y.view_as(top_pred)</code> 는 <code>y</code> 의 크기를 <code>top_pred</code> 크기로 변경 |
| Ⓒ  | <code>sum()</code>          | 합계. 예측과 정답이 일치하는 것들의 개수를 합산                                                                                                                                  |

💡 **view\_as가 왜 필요?** `y`는 (N,), `top_pred`는 (N, 1) → 모양이 달라서 그냥 비교하면 브로드캐스팅으로 N×N 비교가 돼 버림. 모양을 맞추면 뒤 **1:1 비교**하려는 것

#### 17-16. 학습·평가·시간 측정 함수 (코드 6-55 ~ 6-57)

| 함수                                                                | 핵심                                                                                                                                                                                             |
|-------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>train(model, iterator, optimizer, criterion, device)</code> | <code>model.train()</code> → 배치마다 <code>zero_grad</code> → <code>y_pred, _ = model(x)</code> → <code>loss·acc</code> 계산 → <code>backward</code> → <code>step</code> → <code>loss·acc</code> 누적 |
| <code>evaluate(model, iterator, criterion, device)</code>         | <code>model.eval()</code> + <code>torch.no_grad()</code> → <code>loss·acc</code> 만 계산 (갱신 X)                                                                                                   |

| 함수                                            | 핵심                                                                        |
|-----------------------------------------------|---------------------------------------------------------------------------|
| <code>epoch_time(start_time, end_time)</code> | 경과 시간을 분( <code>elapsed_mins</code> )·초( <code>elapsed_secs</code> ) 로 변환 |

- 반환값: `epoch_loss / len(iterator)`, `epoch_acc / len(iterator)` → 배치 평균의 평균

💡 LeNet 때의 `loss.item() * inputs.size(0)` 방식과 다름. 여기선 배치별 평균을 그냥 다시 평균 → 마지막 배치 크기가 다르면 약간 오차가 생김 ( $476 = 128 \times 3 + 92$ )

### 17-17. 모델 학습 (코드 6-58)

| 설정                                                                         | 의미                                                  |
|----------------------------------------------------------------------------|-----------------------------------------------------|
| <code>EPOCHS = 5</code>                                                    | 5 에포크                                               |
| <code>best_valid_loss = float('inf')</code>                                | 최솟값 비교용 초기값 = 무한대                                   |
| <code>time.monotonic()</code>                                              | 모델 훈련의 시작·종료 시간 저장 (되돌아가지 않는 시계 → 시간 측정용)           |
| <code>if valid_loss &lt; best_valid_loss:</code>                           | <code>valid_loss</code> 가 가장 작은 값을 구하고 그 상태의 모델을 저장 |
| <code>torch.save(model.state_dict(), '../chap06/data/VGG-model.pt')</code> | <b>VGG-model.pt</b> 이름으로 저장                         |

### 학습 결과

| Epoch | Epoch Time | Train Loss | Train Acc | Valid Loss | Valid Acc |
|-------|------------|------------|-----------|------------|-----------|
| 01    | 8m 27s     | 0.691      | 54.48%    | 0.693      | 52.83%    |
| 02    | 7m 40s     | 0.690      | 55.38%    | 0.693      | 52.83%    |
| 03    | 7m 34s     | 0.699      | 50.73%    | 0.693      | 60.38%    |
| 04    | 7m 33s     | 0.689      | 53.29%    | 0.693      | 47.17%    |
| 05    | 7m 28s     | 0.700      | 51.52%    | 0.693      | 47.17%    |

- 역시 성능이 좋지 않음 → 데이터셋 이미지 수가 매우 적고, 에포크도 매우 적게 설정됨
- 숫자를 늘리면 성능이 좋아지겠지만, 현재 데이터셋에서 에포크만 늘린다고 좋아지지는 않음 → 데이터도 함께 증가시켜야 함

### 17-18. 테스트 성능 측정 (코드 6-59)

- 저장했던 모델(VGG-model.pt)을 불러와(`model.load_state_dict(torch.load(...))`) 테스트 데이터셋에 대한 성능 측정
- 결과: **Test Loss: 0.693 | Test Acc: 50.00%** (12장 중 6장)
- 역시 결과가 좋지 않음 → 데이터셋을 늘리면 성능이 좋아지지만, 이미지가 늘어난 만큼 훈련 시간이 상당히 길어질 수 있음

### 17-19. 예측 확인 함수 (코드 6-60 `get_predictions`)

| 단계 | 코드                                                              | 의미                                                       |
|----|-----------------------------------------------------------------|----------------------------------------------------------|
| 1  | <code>model.eval(), images = [], labels = [], probs = []</code> | 평가 모드 + 결과 담을 리스트                                        |
| 2  | <code>with torch.no_grad():</code>                              | 기울기 계산 X                                                 |
| 3  | <code>y_pred, _ = model(x)</code>                               | 예측값 (h는 버림)                                              |
| 4  | <code>y_prob = F.softmax(y_pred, dim=-1)</code>                 | 확률로 변환 (dim=-1 = <b>마지막 차원</b> , 여기선 클래스 방향 = dim=1과 같음) |
| 5  | ① <code>top_pred = y_prob.argmax(1, keepdim=True)</code>        | 가장 확률이 높은 클래스 인덱스                                        |
| 6  | <code>images.append(x.cpu())</code> 등                           | 배치별 결과를 <b>CPU로 옮겨</b> 리스트에 저장 (GPU 메모리 절약, 넘파이·시각화용)    |
| 7  | ② <code>torch.cat(images, dim=0)</code>                         | 배치별로 쪼개진 텐서들을 <b>0번 차원(배치 방향)으로 이어 붙여</b> 하나로            |
| 8  | <code>return images, labels, probs</code>                       |                                                          |

💡 **torch.cat** : 128장짜리 묶음, 128장짜리 묶음... 여러 묶음을 **한 줄로 이어 붙여** 큰 묶음 하나로 만드는 것.  
예: (128, 2) + (92, 2) → (220, 2)

## 17-20. 정확하게 예측한 것만 추출 (코드 6-61)

- `get_predictions()`의 반환값을 각각 images, labels, probs에 저장 → 모델이 **정확하게 예측한 이미지**만 추출

| 단계 | 코드                                                                                              | 의미                             |
|----|-------------------------------------------------------------------------------------------------|--------------------------------|
| 1  | <code>images, labels, probs = get_predictions(model, test_iterator)</code>                      | 테스트 데이터 전체 예측                  |
| 2  | ① <code>pred_labels = torch.argmax(probs, 1)</code>                                             | 확률이 가장 높은 클래스 = 예측 레이블         |
| 3  | <code>corrects = torch.eq(labels, pred_labels)</code>                                           | 예측과 정답이 같은지 비교 (True/False 텐서) |
| 4  | ② <code>for image, label, prob, correct in zip(images, labels, probs, corrects):</code>         | 네 리스트를 한 묶음씩 꺼내서               |
| 5  | <code>if correct: correct_examples.append((image, label, prob))</code>                          | 맞힌 것만 (이미지, 정답, 확률) 튜플로 저장     |
| 6  | ③ <code>correct_examples.sort(reverse=True, key=lambda x: torch.max(x[2], dim=0).values)</code> | <b>가장 확실한(최대 확률이 큰) 순서로 정렬</b> |

### ① max vs argmax

- `max` = **최댓값**을 반환, `argmax` = 최댓값을 갖는 **인덱스**를 반환

`x = torch.rand(5, 3)` (5×3 임의의 텐서)

|    | 0열     | 1열            | 2열     |
|----|--------|---------------|--------|
| 0행 | 0.6501 | 0.6177        | 0.5627 |
| 1행 | 0.3163 | 0.5619        | 0.6364 |
| 2행 | 0.8501 | <b>0.9622</b> | 0.4792 |

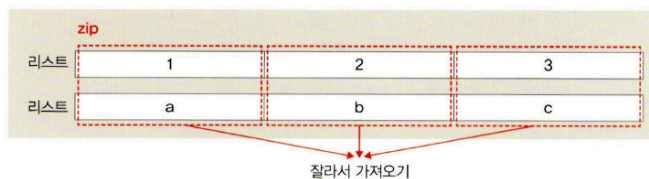
|    | 0열     | 1열     | 2열     |
|----|--------|--------|--------|
| 3행 | 0.1859 | 0.6510 | 0.1824 |
| 4행 | 0.4304 | 0.1170 | 0.8049 |

| 함수                           | 결과                                    |
|------------------------------|---------------------------------------|
| <code>torch.max(x)</code>    | <code>tensor(0.9622)</code> → 가장 높은 값 |
| <code>torch.argmax(x)</code> | <code>tensor(7)</code> → 그 인덱스        |

## ② zip()

- 여러 개의 리스트(혹은 튜플)를 합쳐서 새로운 튜플 타입으로 반환
- 그림 6-18: 리스트 [1, 2, 3]과 [a, b, c]를 같은 위치끼리 잘라서 가져오기

▼ 그림 6-18 zip()의 원리



| 코드                                                                                   | 결과              |
|--------------------------------------------------------------------------------------|-----------------|
| <code>a = [1, 2, 3], b = ['a', 'b', 'c'] → for x, y in zip(a, b): print(x, y)</code> | 1 a / 2 b / 3 c |

- 튜플 = 순서가 있는 데이터의 집합 → 인덱스로 데이터 접근 가능, 하지만 한 번 생성되면 값을 변경할 수 없음

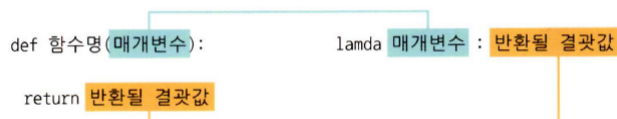
## ③ sort() — `sort(reverse=True, key=lambda x: torch.max(x[2], dim=0).values)`

| 기호 | 파라미터                      | 의미                               | 예시                                                                  |
|----|---------------------------|----------------------------------|---------------------------------------------------------------------|
| Ⓐ  | <code>reverse=True</code> | 내림차순 정렬                          | <code>x = [1, 7, 3, 9] → x.sort(reverse=True) → [9, 7, 3, 1]</code> |
| Ⓑ  | <code>key=</code>         | 정렬할 때 key 값을 기준으로 정렬 (기본값은 오름차순) | 아래 람다                                                               |

## 람다(lambda) 함수

|     | 일반 함수                                        | 람다 함수                             |
|-----|----------------------------------------------|-----------------------------------|
| 형태  | <code>def 함수명(매개변수): + return 반환될 결과값</code> | <code>lambda 매개변수: 반환될 결과값</code> |
| 함수명 | 있음                                           | 없음                                |
| 특징  | 정의 후 재사용                                     | 정의와 동시에 사용                        |

▼ 그림 6-19 일반 함수와 람다 함수



- 예: `y = lambda x: x + 10` → `y(1)` → **11**
- 교재: 함수명이 없고 저장된 변수가 없기 때문에 재사용은 불가능

#### `torch.max(x, dim=0)` 의 의미

`x = torch.randn([4, 4])`

|    | 0열             | 1열            | 2열            | 3열            |
|----|----------------|---------------|---------------|---------------|
| 0행 | <b>-0.3027</b> | -0.3185       | 0.4117        | 0.4915        |
| 1행 | -3.1112        | <b>1.3952</b> | -0.2522       | -0.5341       |
| 2행 | -0.5381        | -0.2026       | -1.0494       | -1.2712       |
| 3행 | -0.3689        | -1.8523       | <b>2.0229</b> | <b>0.5725</b> |

| 반환값                                | 결과                                        |
|------------------------------------|-------------------------------------------|
| <code>max_elements</code> (values) | tensor([-0.3027, 1.3952, 2.0229, 0.5725]) |
| <code>max_idx</code> (indices)     | tensor([0, 1, 3, 3])                      |

- `torch.max(x, dim=0)` → **값과 인덱스 두 개**를 반환 (dim을 안 주면 값만 반환)
- 교재 표현: dim=0(행을 기준)으로 최댓값을 가져옴

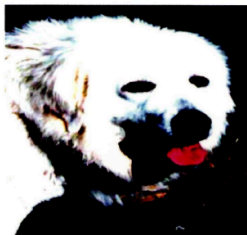
💡 **dim=0 헛갈리지 않기**: dim=0 방향(위→아래, 행을 따라)으로 훑어서 **열마다** 최댓값 하나씩 → 결과 길이 = 열 개수(4). "0번 차원을 없앤다(압축한다)"로 기억하면 편함

💡 **정리**: `key=lambda x: torch.max(x[2], dim=0).values` → x는 (이미지, 정답, 확률) 튜플 → `x[2]` = 확률 벡터 → 그중 최댓값 = **모델의 확신도** → 확신도가 높은 순(reverse=True)으로 정렬

#### 17-21. 이미지 출력용 정규화 (코드 6-62 `normalize_image`)

- 앞에서 이미지 크기 조정·정규화 등 전처리를 거쳤기 때문에 이 상태 그대로 출력하면 **색상이 왜곡됨** (그림 6-20: 하얀 개가 과하게 하얗고 검게 깨진 이미지)
- 본래 이미지(그림 6-21)로 출력하려면 아래 전처리 필요

▼ 그림 6-20 본래 이미지 색상이 왜곡됨



| 단계 | 코드                                                            | 의미                                            |
|----|---------------------------------------------------------------|-----------------------------------------------|
| 1  | <code>image_min = image.min(), image_max = image.max()</code> | 이미지의 최솟값·최댓값                                  |
| 2  | <code>image.clamp_(min=image_min, max=image_max)</code>       | 주어진 최소(min)-최대(max) <b>범주 안에</b> 이미지가 위치하도록 함 |

| 단계 | 코드                                                                       | 의미                                               |
|----|--------------------------------------------------------------------------|--------------------------------------------------|
| 3  | ① <code>image.add_(-image_min).div_(image_max - image_min + 1e-5)</code> | 최솟값을 빼고(→ 최소 0) 범위로 나눔(→ 최대 1) = <b>0~1로 맞추기</b> |

▼ 그림 6-21 본래 이미지



💡 **왜 왜곡될까?** Normalize 후 픽셀 값은 (값 - 평균) / 표준편차라서 **음수나 1보다 큰 값**이 섞여 있음 → matplotlib은 0~1 범위를 기대하니 범위 밖은 잘려서 색이 깨짐 → 다시 **0~1로 눌러 맞추는 것**(min-max 정규화)

정규화된 값 =  $(x - \min) / (\max - \min + 0.00001)$

- `+ 1e-5` (0.00001): 모든 픽셀이 같은 값이면  $\max - \min = 0 \rightarrow$  **0으로 나누는 에러 방지용** 아주 작은 수

#### ① torch.add vs torch.add\_

| 코드                                                    | 결과               | <code>x is y</code> |
|-------------------------------------------------------|------------------|---------------------|
| <code>x = torch.tensor([1, 2]) , y = x.add(10)</code> | tensor([11, 12]) | <b>False</b>        |
| <code>y = x.add_(10)</code>                           | tensor([11, 12]) | <b>True</b>         |

|       | <code>torch.add</code> | <code>torch.add_</code>  |
|-------|------------------------|--------------------------|
| 결과 저장 | 새로운 메모리 공간을 할당해 저장     | 새 공간 할당 없이 기존 메모리의 값을 대체 |
| 원본 x  | 그대로                    | 바뀜                       |

- 메서드에 **\_ (언더바)** 표시가 있다면 (예: `torch.clamp_()`) → 기존 메모리 공간에 있는 값을 **새로운 값으로 대체**하겠다는 의미

💡 AlexNet의 `ReLU(inplace=True)` 와 같은 개념! **\_ = inplace 연산**(덮어쓰기)

#### 17-22. 정확히 예측한 이미지 출력 함수 (코드 6-63 `plot_most_correct`)

- 모델이 정확하게 예측한 이미지만 출력 + 출력 방법 정의

| 단계 | 코드                                                                         | 의미                                             |
|----|----------------------------------------------------------------------------|------------------------------------------------|
| 1  | <code>rows = int(np.sqrt(n_images)) , cols = int(np.sqrt(n_images))</code> | 격자 크기 ( <code>np.sqrt</code> = 제곱근 = 0.5 거듭제곱) |
| 2  | <code>fig = plt.figure(figsize=(25, 20))</code>                            | 그림판 생성                                         |

| 단계 | 코드                                                                                    | 의미                                               |
|----|---------------------------------------------------------------------------------------|--------------------------------------------------|
| 3  | <code>ax = fig.add_subplot(rows, cols, i+1)</code>                                    | 출력하려는 그래프 개수만큼 subplot 생성                        |
| 4  | <code>image, true_label, probs = correct[i]</code>                                    | i번째 정답 예시 꺼내기                                    |
| 5  | ① <code>image = image.permute(1, 2, 0)</code>                                         | 축 순서 변경 (C, H, W) → <b>(H, W, C)</b>             |
| 6  | <code>true_prob = probs[true_label]</code>                                            | 정답 클래스에 대한 확률                                    |
| 7  | <code>correct_prob, correct_label = torch.max(probs, dim=0)</code>                    | 가장 높은 확률과 그 클래스                                  |
| 8  | <code>true_class = classes[true_label], correct_class = classes[correct_label]</code> | 인덱스 → 클래스 이름                                     |
| 9  | <code>if normalize: image = normalize_image(image)</code>                             | 본래 이미지대로 출력하기 위해 <code>normalize_image</code> 호출 |
| 10 | <code>ax.imshow(image.cpu().numpy())</code>                                           | 텐서 → 넘파이로 바꿔서 출력                                 |
| 11 | <code>set_title(f'true label: ... \n pred label: ...')</code>                         | 정답·예측 클래스와 확률(소수점 3 자리)을 제목으로                    |
| 12 | <code>ax.axis('off'), fig.subplots_adjust(hspace=0.4)</code>                          | 축 제거, 위아래 간격 조정                                  |

#### ① permute — 축 변경

| 코드                                                          | 결과                                                |
|-------------------------------------------------------------|---------------------------------------------------|
| <code>x = torch.tensor([[1, 2, 3], [4, 5, 6]])</code> (2×3) | [[1, 2, 3], [4, 5, 6]]                            |
| <code>x.permute(1, 0)</code> → 차원(축) 0과 1을 바꿈               | [[1, 4], [2, 5], [3, 6]] (3×2) → <b>행과 열이 변경됨</b> |

💡 **왜 permute(1, 2, 0)?** 파이토치 이미지는 **(채널, 높이, 너비)**, matplotlib은 **(높이, 너비, 채널)** 을 원함 → "지금 1번(H)을 0번 자리로, 2번(W)을 1번 자리로, 0번(C)을 마지막 자리로"

- ToTensor가 (H, W, C) → (C, H, W)로 바꿨던 걸 **되돌리는 것**

#### 17-23. 예측 결과 이미지 출력 (코드 6-64)

- `classes = test_dataset.classes` → ImageFolder가 폴더명으로 만든 클래스 목록 (['Cat', 'Dog'])
- `N_IMAGES = 5` → `plot_most_correct(correct_examples, classes, N_IMAGES)`
- 그림 6-22: 정확하게 예측한 이미지 **4장** (전부 개), 제목은 `true label: Dog (0.501) / pred label: Dog (0.501)`

♥ 그림 6-22 테스트 데이터셋에 대한 VGG 모델의 예측 결과



💡 왜 5장이 아니라 4장?  $\text{rows} = \text{cols} = \text{int}(\sqrt{5}) = \text{int}(2.23...) = 2 \rightarrow 2 \times 2 = 4$ 칸만 그림  
 💡 확률이 전부 **0.501**  $\rightarrow$  0.5를 아주 살짝 넘겨서 "맞힌" 것. 모델이 확신하고 맞힌 게 아니라 거의 찍었는데 Dog 쪽으로 기운 것 (앞의 Test Acc 50%와 같은 맥락)

- 결과의 정확도가 높지 않음  $\rightarrow$  데이터셋을 늘려 준다면 정확도가 높아질 것

## 18. 6.1.4 GoogLeNet

### 18-1. GoogLeNet이란?

- 주어진 하드웨어 자원을 최대한 효율적으로 이용하면서 학습 능력은 극대화할 수 있는 깊고 넓은 신경망
- 깊고 넓은 신경망을 위해 인셉션(inception) 모듈을 추가  $\rightarrow$  그래서 인셉션이라고도 불림

### 18-2. 왜 필요했나? — 깊고 넓은 네트워크의 문제

- ImageNet 같은 대회 참여나 서비스 제공을 위해선 대용량 데이터를 학습해야 함
- 심층 신경망에서 계층이 넓고(뉴런이 많고) 깊으면(계층이 많으면)  $\rightarrow$  인식률은 좋아지지만 문제 발생 (특히 합성곱 신경망에서 자주 나타남)

| 문제                                     | 설명                                       |
|----------------------------------------|------------------------------------------|
| 과적합                                    | 파라미터가 너무 많아 훈련 데이터를 외워 버림                |
| 기울기 소멸 문제 (vanishing gradient problem) | 층이 깊어질수록 역전파되는 기울기가 점점 작아져 앞쪽 층이 학습되지 않음 |
| 학습 시간 지연                               | 연산량이 많아 오래 걸림                            |
| 연산 속도                                  | 하드웨어 자원 부담                               |

$\rightarrow$  GoogLeNet(인셉션)으로 이러한 문제를 해결할 수 있다고 생각하면 됨

### 18-3. 핵심 해결책: 희소 연결 (sparse connectivity)

|    | 밀집 연결 (dense)                           | 희소 연결 (sparse connectivity)   |
|----|-----------------------------------------|-------------------------------|
| 의미 | 합성곱·풀링·완전연결층들이 서로 정교하고 뽀뽀하게 연결 (기존 CNN) | 관련성(correlation)이 높은 노드끼리만 연결 |

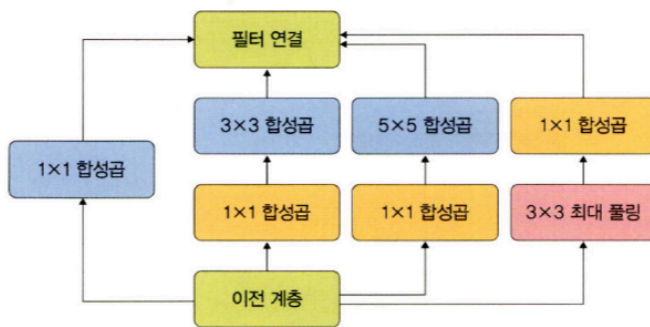
|     |               |                             |
|-----|---------------|-----------------------------|
|     | 밀집 연결 (dense) | 희소 연결 (sparse connectivity) |
| 연산량 | 많음            | 적어짐                         |
| 과적합 | 발생하기 쉬움       | 해결 가능                       |

#### 18-4. 인셉션 모듈

- 특징을 효율적으로 추출하기 위해 **1×1, 3×3, 5×5 합성곱 연산을 각각(동시에) 수행**
- 3×3 최대 풀링은 입력과 출력의 **높이와 너비가 같아야** 하므로 풀링 연산에서는 드물게 **패딩을 추가**해야 함

구조: 이전 계층 → **4갈래로 동시에 처리** → 필터 연결(filter concatenation)

▼ 그림 6-23 GoogLeNet의 인셉션 모듈



| 갈래 | 연산                                              |
|----|-------------------------------------------------|
| ①  | 1×1 합성곱                                         |
| ②  | 1×1 합성곱 → 3×3 합성곱                               |
| ③  | 1×1 합성곱 → 5×5 합성곱                               |
| ④  | 3×3 최대 풀링(max pooling) → 1×1 합성곱(convolutional) |

#### 💡 왜 여러 크기를 동시에?

- 사진 속 개가 화면 가득일 수도, 구석에 작게 있을 수도 있음 → 어떤 크기의 돋보기가 맞을지 모르니 **작은 (1×1)·중간(3×3)·큰(5×5) 돋보기를 한꺼번에** 대 보고, 결과를 **모두 합쳐서**(필터 연결) 다음 층에 넘김
- VGG가 "같은 크기 돋보기를 **깊게** 쌓기"였다면, GoogLeNet은 "다른 크기 돋보기를 **넓게** 펼치기"

#### 💡 3×3, 5×5 앞에 왜 1×1 합성곱이?

- 1×1 합성곱은 공간(높이·너비)은 그대로 두고 **채널 수만 줄이는** 역할 (병목, bottleneck)
- 예: 채널 256짜리를 바로 5×5로 처리하면 연산이 엄청 큼 → 먼저 1×1로 채널을 64로 줄인 뒤 5×5를 하면 **연산량이 크게 감소**
- 이게 "하드웨어 자원을 효율적으로 쓰면서" 넓게 만들 수 있는 비결

#### 💡 왜 풀링에 패딩까지 넣어서 크기를 맞추나?

- 마지막에 4갈래 결과를 **채널 방향으로 이어 붙여야**(torch.cat의 dim=1과 같은 개념) 하는데, 그러려면 4갈래 모두 **높이·너비가 같아야** 함 → 그래서 풀링도 크기가 줄지 않도록 패딩 + 스트라이드 1로 맞춤